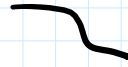


CSDC0306

Machine learning



Amritpal Singh
CSED, NIT-J



What is this subject??

Course Contents**Unit 1: Introduction to Machine Learning and Mathematical Foundations**

Introduction to ML, Definitions, Applications, Types (Supervised, Unsupervised, Reinforcement Learning), History and Evolution of ML, Comparison with Traditional Programming ML Pipeline: Data Collection, Processing, Model Selection, Training, Evaluation.

✓ Linear Algebra: Vectors, Matrices, Eigenvalues & Eigenvectors, Optimization Techniques: Gradient Descent, Convex Optimization, Loss Functions: MSE, Cross-Entropy, Hinge Loss, Evaluation Metrics: Accuracy, Precision, Recall, F1 Score, ROC Curve.

Unit 2: Supervised Learning

Regression Models, Linear Regression: Model, Cost Function, Gradient Descent, Multiple Linear Regression & Polynomial Regression Ridge & Lasso Regression
Classification Models Logistic Regression, Decision Trees: Splitting Criteria (Gini Index, Entropy), Pruning of decision tree, Multivariate Trees, Hypothesis space search in decision tree learning, Inductive bias in decision tree learning, Issues in decision tree learning, Random Forests & Ensemble Learning (Bagging, Boosting, Stacking), Support Vector Machines (SVM): Kernel Trick, Hyperplane Concept, k-Nearest Neighbors (k-NN) Algorithm, Naïve Bayes Classifier, Bayesian belief networks Case Study: Predicting House Prices and Spam Classification, Artificial Neural Network (ANN).

Unit 3: Unsupervised Learning

Introduction to Unsupervised Learning, Clustering: K-Means, k-medoid Hierarchical Clustering, DBSCAN, Clustering & Gaussian Mixture Models (GMM), Case Study: Customer Segmentation & Anomaly Detection.

Unit 4: Reinforcement Learning

Overview of Reinforcement Learning: Elements of Reinforcement Learning, Markov Decision Process (MDP) and its Components, Exploration vs. Exploitation Trade-off, Dynamic Programming for RL: Bellman Equations, Value Iteration and Policy Iteration, Finding Optimal Policies, Model-Free Reinforcement Learning, Temporal Difference (TD) Learning: SARSA (On-Policy Learning) Q-Learning (Off-Policy Learning), REINFORCE Algorithm and Actor-Critic Methods (A2C, A3C), Exploration Strategies: ϵ -Greedy, Softmax Exploration, Applications of Reinforcement Learning.

Course Outcomes

- CO1: Understand fundamental machine learning concepts and their real-world applications.
CO2: Implement and analyze machine learning algorithms, including regression, classification, clustering, and reinforcement learning.
CO3: Evaluate machine learning models using metrics like accuracy, precision, recall, and F1-score.
CO4: Apply machine learning techniques to real-world problems, optimize models, and interpret results for better decision-making.

Course Outcomes	Programme Outcomes													
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2
CO1	3	2		1						1			1	2
CO2	3	3	3	3	2								3	1
CO3	2	2	1	3						1	3	2	2	2
CO4	2	2	2	2					1		3	2		1

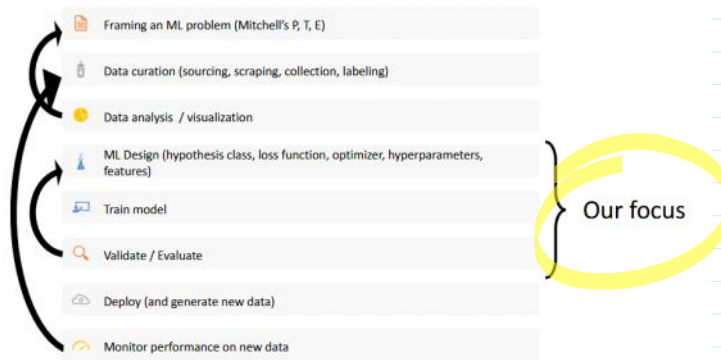
1 – Low, 2 – Medium, 3 – High

Recommended Books

- Mohri, Mehryar, Afshin Rostamizadeh, and Ameet Talwalkar. Foundations of Machine Learning. 2nd ed., MIT Press, 2018.
- Sammut, Claude, and Geoffrey I. Webb, editors. Encyclopedia of Machine Learning and Data Mining. 3rd ed., Springer, 2023.
- Witten, Ian H., Eibe Frank, Mark A. Hall, and Christopher J. Pal. Data Mining: Practical Machine Learning Tools and Techniques. 5th ed., Morgan Kaufmann, 2024.
- Müller, Andreas C., and Sarah Guido. Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media, 2016.
- Bishop, Christopher M. Pattern Recognition and Machine Learning. Springer, 2013.

****show road map**

Machine Learning Workflow



Why is this subject??

- Human being always dreamt of creating machines with human like-
- Some example- **Robots in manufacturing**, **mining**, **agriculture**, **space**, **ocean exploration** and **health science** etc.
- However these machines are **enslaved by commands**.
- The goal of robotics and system science is to **cast intelligence into machine**.
 - to create intelligent machine which can emulate human intelligence.

Why is Machine Learning Important?

Some tasks cannot be defined well, except by examples (e.g., recognizing people).

Relationships and correlations can be hidden within large amounts of data. Machine Learning/Data Mining may be able to find these relationships.

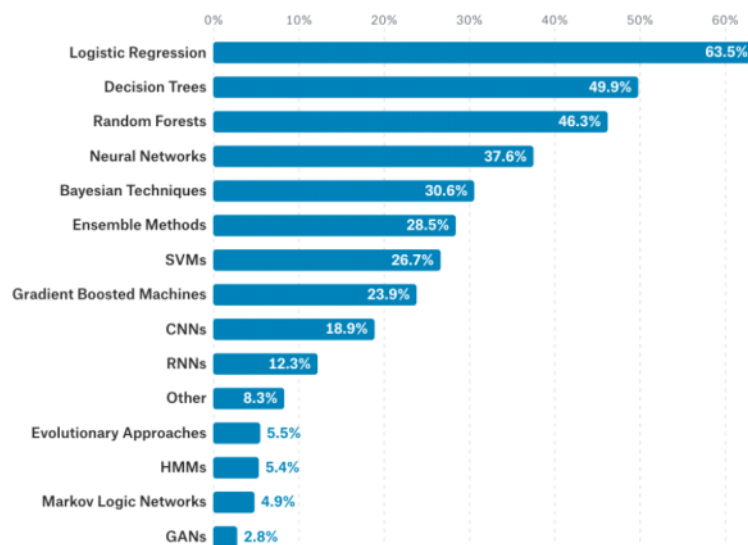
Human designers often produce machines that do not work as desired in the environments in which they are used.

The amount of knowledge available about certain tasks might be too large for explicit encoding by humans (e.g., medical diagnostic).

Environments change over time.

New knowledge about tasks is constantly being discovered by humans. It may be difficult to continuously re-design systems "by hand".

2017 Kaggle survey of data science and ML practitioners: what data science methods do you use at work?



How this subject is applied in real life?

$$\begin{cases} 2x - 3y + z = -1 \\ x - y + 2z = -3 \\ 3x + y - z = 9 \end{cases}$$

Equation solving



Delaunay triangulation



Shortest path



Minimum spanning tree



Face recognition



Autonomous driving



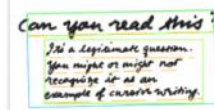
Spam filtering



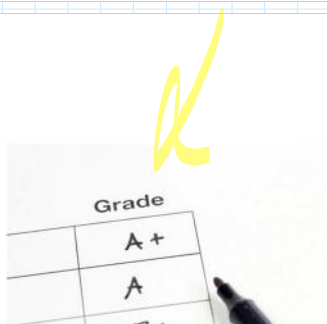
Recommender systems

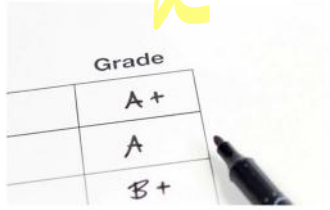
Applications of machine learning

- Self-driving cars
- Face recognition
- Handwriting recognition
- Amazon product recommendation
- Spam filtering
- Automatic translation
- Speech recognition
- ...

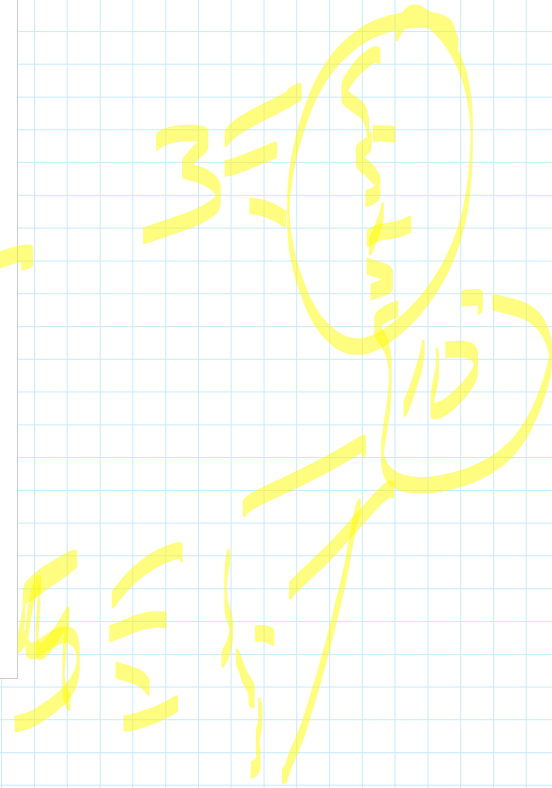


What is there for me??





S. No.	Evaluation Elements	Weightage (%)
1	Mid Sem Exam	30
2	End Semester Exam	50
3	Continuous Evaluation (May include assignments/ Tutorials/Quizzes)	20



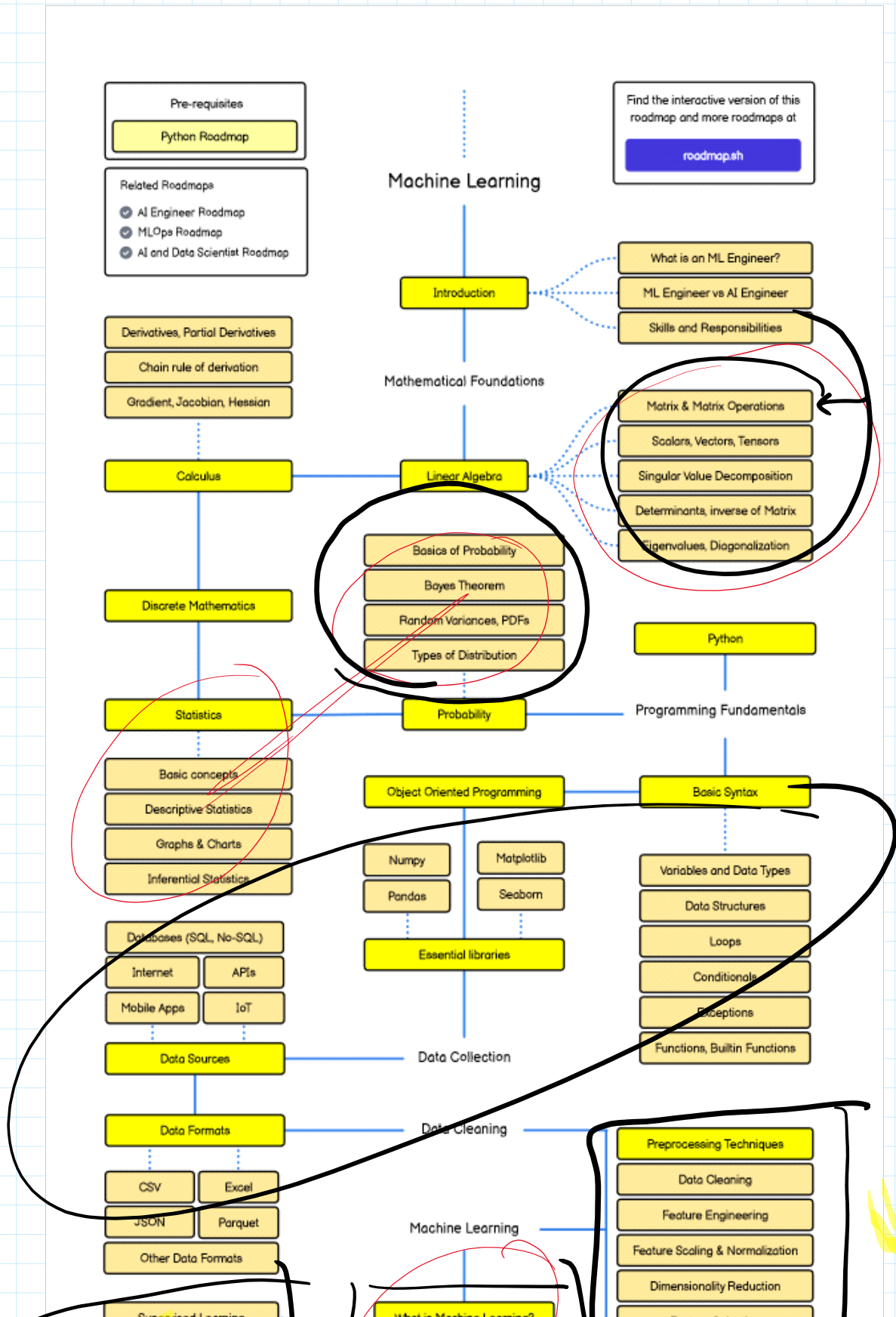
ROAD MAP

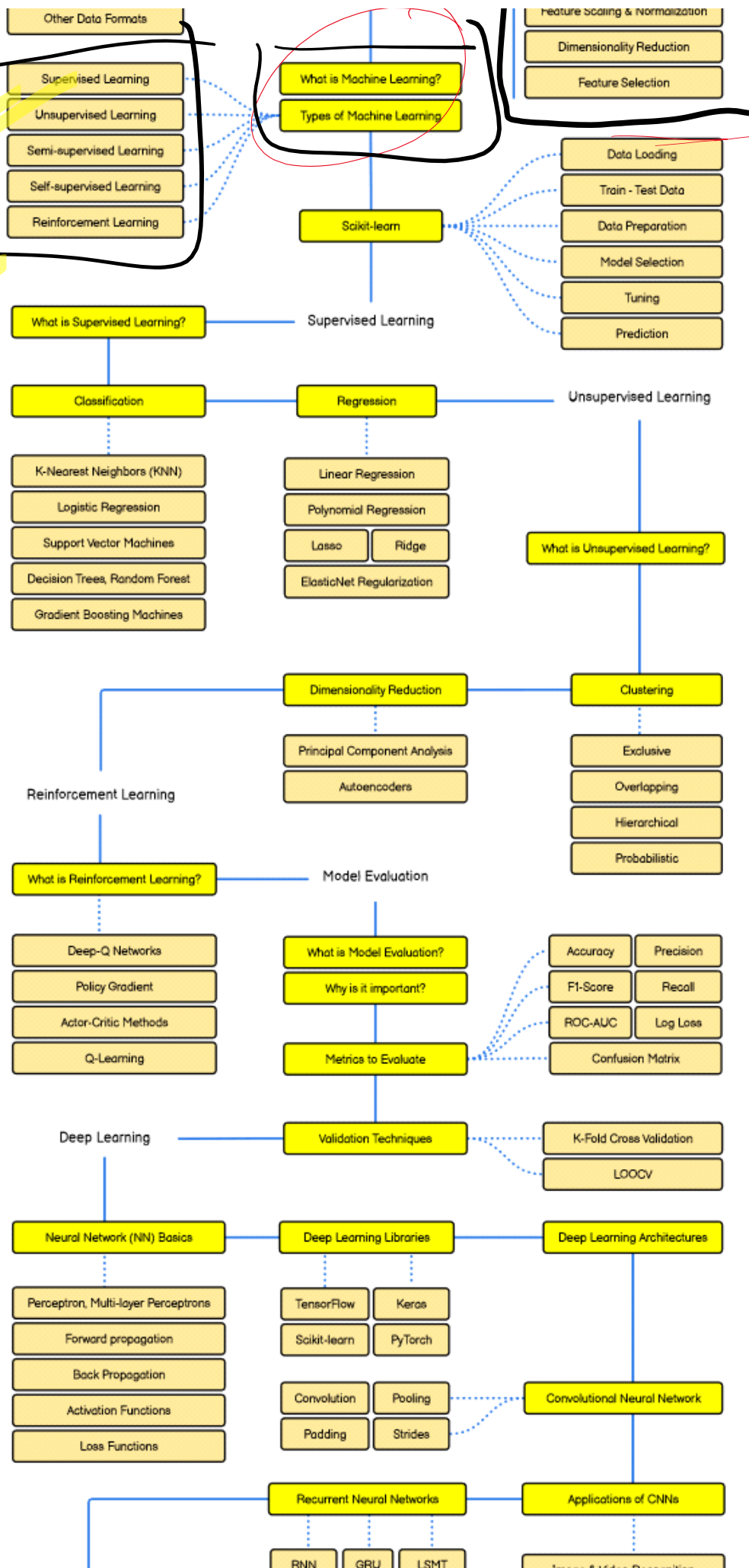
18 January 2026 01:51 PM

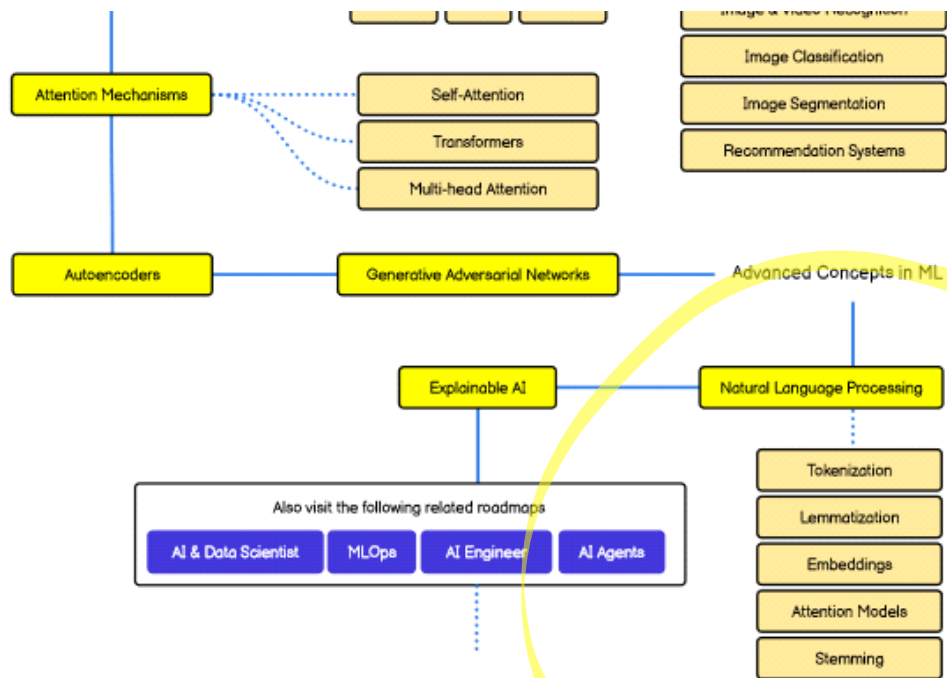


ROAD MAP
machine-l...

<https://roadmap.sh/machine-learning?fl=1>







History of machine learning

(We only review a small subset of events and influential methods/techniques)

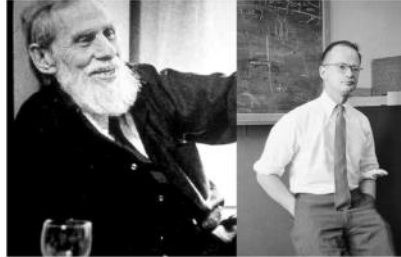
- 1943: First mathematical model of neural networks
 - Warren McCulloch (left) and Walter Pitts (right)

BULLETIN OF
MATHEMATICAL BIOPHYSICS
VOLUME 5, 1943

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

FROM THE UNIVERSITY OF ILLINOIS, COLLEGE OF MEDICINE,
DEPARTMENT OF PSYCHIATRY AT THE ILLINOIS NEUROPSYCHIATRIC INSTITUTE,
AND THE UNIVERSITY OF CHICAGO



13

First artificial neuron model

Dmitrijus Bogdanovas Follow Sep 8, 2023

Who suggested it?

Was it Walter Pitts? No. Was it Warren McCulloch? No.

Were it Walter Pitts AND Warren McCulloch? No.

It was Nikolay Rashevsky in 1933, in paper "Outline of a psycho-mathematical theory of excitation and inhibition".

Ten years later, in 1943, his student W. Pitts and W. McCulloch released much more well-known paper "A logical calculus of the ideas immanent in nervous activity", which was published in the Bulletin of Mathematical Biophysics, journal founded by N. Rashevsky. Pitts have mentioned nothing about his

✓ Nikolay Rashevsky (1933)

- Nikolay Rashevsky proposed one of the earliest mathematical models of a neuron in 1933.
- His paper "Outline of a Psycho-Mathematical Theory of Excitation and Inhibition" introduced continuous, differential-equation-based models to describe neural excitation and inhibition.
- This work is widely regarded as the first mathematical/neurobiological neuron model.

✓ McCulloch & Pitts (1943)

- Warren McCulloch and Walter Pitts proposed the first artificial neuron model used in artificial intelligence and neural networks.
- Their neuron was a binary, logical abstraction, suitable for computation and digital implementation.
- This model directly influenced modern ANNs, perceptrons, and deep learning.

🔍 Correct way to state it (exam-safe)

- First mathematical neuron model:
 - 👉 Nikolay Rashevsky (1933)
- First artificial/computational neuron model:
 - 👉 McCulloch & Pitts (1943)

Feature Field **Rashevsky's Model (1933)**
Mathematical Biophysics

Mathematics Used **continuous** math (differential equations) to model the physical process of excitation.

Primary Goal To explain the **biological physics** of how a cell reacts to stimuli.

Legacy Father of Mathematical Biology.

McCulloch-Pitts Model (1943)
Computational Logic / AI

Used **discrete** math (symbolic logic) to model the neuron as a switch.

To prove that neurons could perform **logical computation** (like a computer).

Founders of Artificial Neural Networks.

History of machine learning

- 1956: Championship-level **computer checkers game**

- Not explore every possible path
- Find **optimal move by measuring chances of winning**
- Mechanisms to continuously improve
 - Remember previous moves
 - Compare with chances of winning

Arthur Samuel is the first person to come up with and popularize the term "**machine learning**".



Arthur Samuel and IBM 700

14

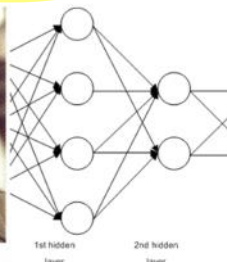
History of machine learning

- 1965: **First Deep Neural Network**

- Foundations for nowadays' most powerful algorithms
- First multi-layer perceptron
- Alexey Ivakhnenko is considered the father of deep learning
- Not popular until around 2010
 - Limited computing power
 - Lack of annotated data



O.I. Ivakhnenko (1967 p.)

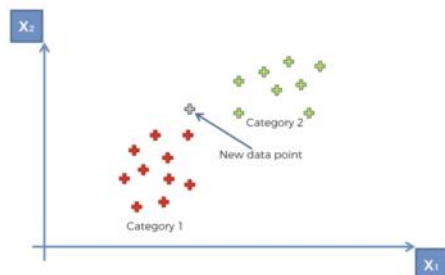


15

History of machine learning

- 1967: Nearest Neighbor Pattern Classification

- Basic idea: It assigns to an unclassified sample point the classification of the nearest of a set of previously classified points.



Thomas Cover (bottom) and Peter Hart (top)

16

History of machine learning

- 1990: Boosting algorithm
 - Aims to enhance predicting power
 - No single strong model is proposed
 - Combine the predictions of many weak models
 - Using averages or voting



Figure 1: Error rate vs. number of weak classifiers. The graph shows that the error rate decreases as the number of weak classifiers increases, illustrating the power of ensemble methods like boosting.

Figure 2: Error rate vs. number of weak classifiers. The graph shows that the error rate decreases as the number of weak classifiers increases, illustrating the power of ensemble methods like boosting.

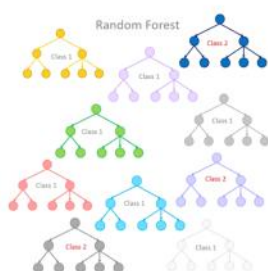
The Strength of Weak Learnability.
Robert Schapire (top) and Yoav Freund (bottom)



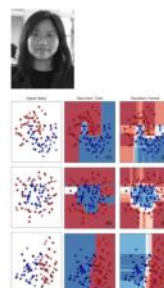
17

History of machine learning

- 1995: Random decision forests
 - Creates and merges decisions from individual tree structures into a "forest"
 - Significantly improves its accuracy and decision-making



Random Decision Forests
Leo Breiman
1999



18

History of machine learning

- 2009: ImageNet Large Scale Visual Recognition Challenge
 - 1K object categories
 - > 14 million manually annotated images
 - Crowdsourced annotation (otherwise 19 years)



[Imagenet: A large-scale hierarchical image database](#)



Fei-Fei Li

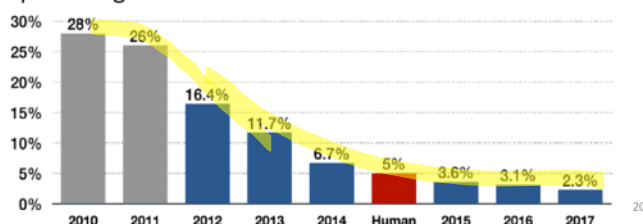
19

History of machine learning

• 2009: ImageNet Large Scale Visual Recognition Challenge

- 1K object categories
- > 14 million manually annotated images
- Crowdsourced annotation
- Error rate: 28% (2010), 16% (2012, AlexNet) ...
- The start of a "deep learning revolution"

- Deep learning revolution
- Transformed the AI industry



History of machine learning

• 2014: Generative adversarial networks (GAN)

- Teaches AI how to generate new data based on training set
- Two network opposing each other
 - Generator vs Discriminator



21

NIPS
AA

History of machine learning

• 2015: DeepMind's AlphaGo

- The first AI to beat a professional Go player

• 2017: Waymo launches autonomous taxis

• 2021: DeepMind's AlphaFold

- Reveals human protein structures

• 2022: ChatGPT



22

2022

- **ChatGPT (OpenAI)** popularized Large Language Models (LLMs) for human-like interaction.
- **Diffusion models** (DALL-E 2, Stable Diffusion) revolutionized text-to-image generation.

2023

- **GPT-4** introduced multimodal learning (text + images).
- **Open-source LLMs** (LLaMA) boosted academic research and customization of models.

2024

- Focus shifted to **efficient ML** (LoRA, PEFT, model compression).
- ML increasingly applied to **climate science, healthcare, and engineering design**.

2025-2026 (Current Trend)

- Emergence of **agentic AI systems** combining LLMs with planning and tool usage.
- Companies shipped multimodal assistants (text+vision+tooling) and agent frameworks that plan and call tools—moving from chatbots to autonomous collaborators.
- Shift toward **general-purpose, autonomous, and responsible AI systems**.

What is learning?

"The activity or process of gaining knowledge or skill by studying, practicing, being taught, or experiencing something."

Human Intelligence

- Posses complex sensory mechanism, control, effective (emotional process), and cognitive (thought process) aspect of information processing and decision making.
- The process is supported by Biological neurons, over one hundred billions in numbers, in our central Nervous system (CNS).
- CNS acquires information from external environment through various natural sensory mechanism. Integrate the information and provide appropriate interpretation through cognitive computing.

Human Intelligence and Computers

- Traditionally, computers have been mainly used for storage and processing of numerical data.
- Can we process some cognitive function such as learning, remembering, reasoning, perceiving etc.
- We must to develop new tools and hardware that must deal with cognitive learning .
- The process of cognitive thinking not necessarily follow the mathematical laws.
 - Is there any cognitive mathematics ?
- The computing methods based on neural networks will be able to provide a thinking machine.

What is Machine Learning



Machines are taking over!

What is machine learning?



- Ways people have tried to define machine learning
 - *A field of study that gives computers the ability to learn without being explicitly programmed* - **Arthur Samuel**

Known for

- Pioneer in Machine Learning
- Development of TeX project (with Donald Knuth)
- Checkers-playing program



What is machine learning?

- Ways people have tried to define machine learning
 - A field of study that gives computers the ability to learn without being explicitly programmed - **Arthur Samuel**
 - A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on T , as measured by P , improves with experience E . - **Tom Mitchell**

Known for

- contributions to ML and AI
- Author of textbook "Machine Learning"



5

What is machine learning?

- Ways people have tried to define machine learning
 - A field of study that gives computers the ability to learn without being explicitly programmed - **Arthur Samuel**
 - A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E . - **Tom Mitchell**
 - Machine learning is the study of computer algorithms that can improve automatically through experience and by the use of data. Machine learning algorithms build a model based on sample data, known as training data, in order to make predictions or decisions without being explicitly programmed to do so. - **Wikipedia**

6

What is machine learning?



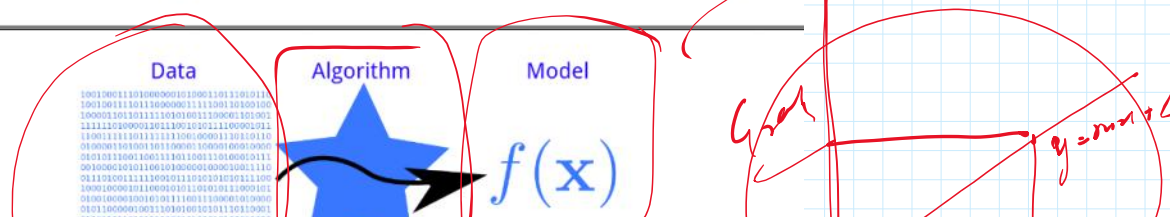
A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on T , as measured by P , improves with experience E . - **Tom Mitchell**

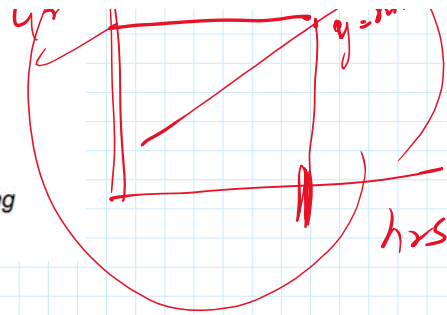
Suppose we feed a learning algorithm a lot of historical weather data, and have it learned to predict the weather. What would be a reasonable choice for P ?

- The process of the algorithm examining a large amount of historical weather data.
- The weather prediction task.
- The probability of it correctly predicting a future date's weather.
- None of these.

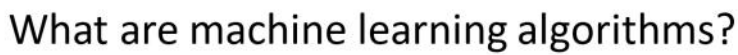
7

But Seriously, What is Machine Learning?





| A. Jung, "Machine Learning: The Basics," Springer, Singapore, 2022



$$\begin{cases} 2x - 3y + z = -1 \\ x - y + 2z = -3 \\ 3x + y - z = 9 \end{cases}$$



Equation solving

Delaunay triangulation

Shortest path

Minimum spanning tree



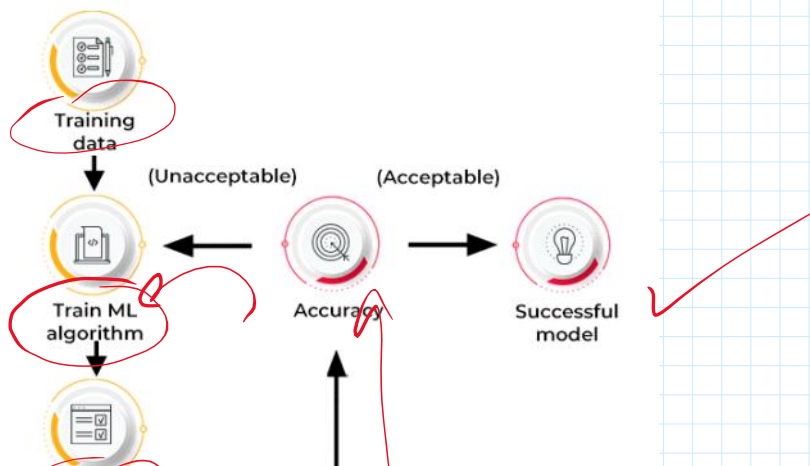
Face recognition

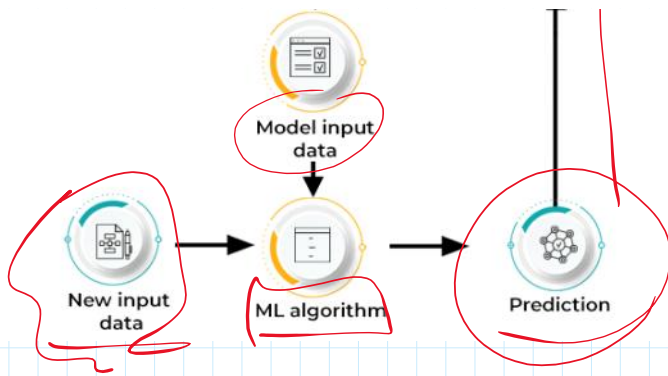
Autonomous driving

Spam filtering

Recommender systems

HOW DOES MACHINE LEARNING WORK?







ML vs AI

Though “machine learning” and “artificial intelligence” are often used interchangeably, they are not quite synonymous.

In short: all machine learning is AI, but not all AI is machine learning.

EXAMPLE

AI	ML
Email auto-reply using fixed templates	Spam filter that learns from emails
Traffic signal with fixed timing	Traffic signal that adapts from traffic data

Machine Learning

- Machine Learning is a particularly interesting technique because it represents a paradigm shift within AI

Traditional AI techniques



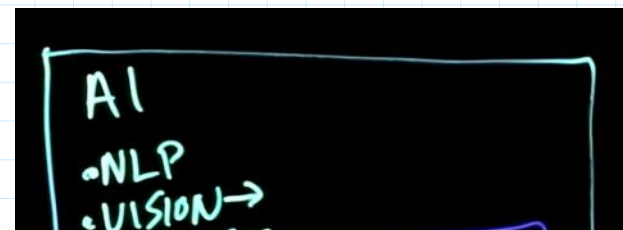
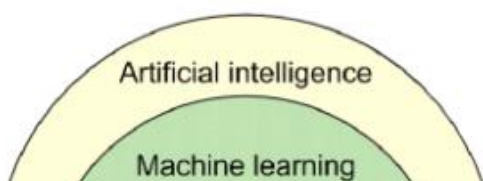
- Static** – hard-coded set of steps and scenarios
- Rule Based** – expert knowledge
- No generalization** – handling special cases is difficult

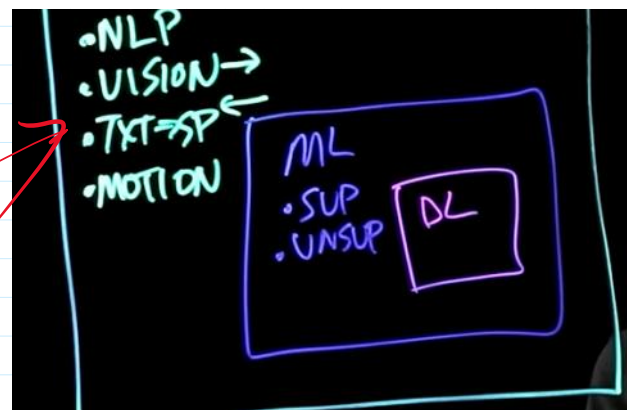
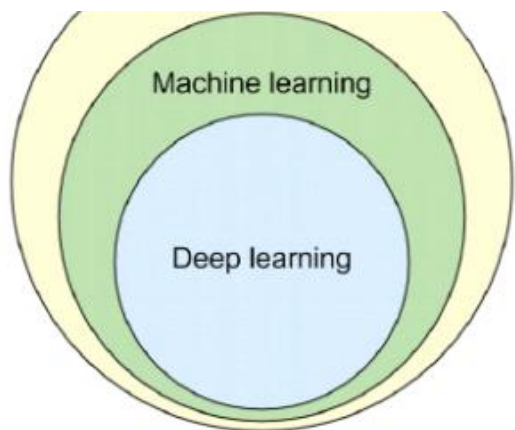
Machine Learning



- Dynamic** – evolves with data, finds new patterns
- Data driven** – discovers knowledge
- Generalization** – adapts to new situations and special cases

Artificial intelligence VS. Machine learning VS. Deep learning





AI

IF attendance < 75%

→ Defaulter

ELSE

→ Allowed

- Rules written by humans
- No learning

ML

Inputs: CGPA, attendance, coding practice, internships

↓
ML Algorithm (trained on previous batches)

↓
Prediction: High / Medium / Low placement chance

- Learns patterns from data
- Needs **features selected by humans**



DL

Raw Data:

- Resume text (PDF)
- GitHub activity
- Coding test logs
- Interview video/audio

- Uses **multi-layer neural networks**
- No manual feature extraction
- Works best with **large data**

- GitHub activity
- Coding test logs
- Interview video/audio

↓

Deep Neural Network

↓

Output:

- Shortlist / Reject
- Confidence score
- Skill match score

- Uses ~~multi-layer neural networks~~
- No manual feature extraction
- Works best with **large data**

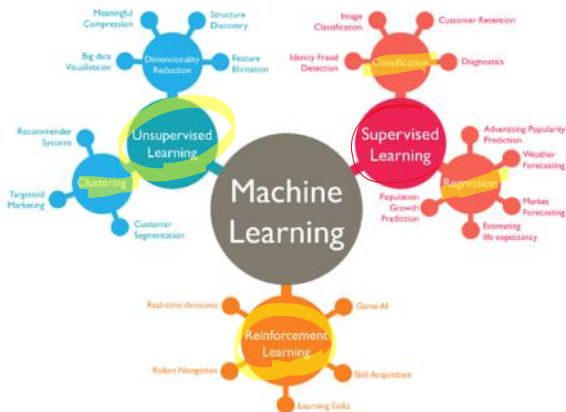
Types of Learning

- **Supervised learning**
 - **Input:** Examples of inputs and outputs
 - **Output:** Model that predicts unknown output given a new input
- **Unsupervised learning**
 - **Input:** Examples of some data (no "outputs")
 - **Output:** Representation of structure in the data
- **Reinforcement learning**
 - **Input:** Sequence of interactions with an environment
 - **Output:** Policy that performs a desired task

Types of Machine Learning



0 0 0 0 0
1 1 1 1 1
2 2 2 2 2



Supervised learning

- Learn from both labeled inputs and desired outputs
 - Almost all applications of deep learning that are in the spotlight these days belong in this category: optical character recognition, speech recognition, image classification/segmentation, object detection, and language translation
- Good at
 - **Regression:** map input variables to a continuous function and predict values
 - Given sizes (and energy labels, ages, distance to city center) of houses, predict their price
 - Given a picture of a person, predict his/her age
 - **Classification:** map input variables into discrete categories
 - Given a patient with a tumor, predict whether the tumor is malignant or benign
 - Spam mail detection

Example of regression



Example of classification



Supervised learning

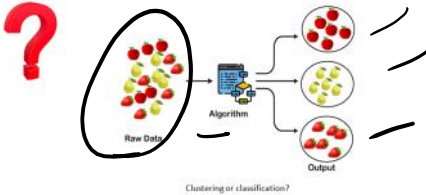
- ?
- Exercise 1: Which is regression, and which is classification?
 - **Problem 1:** Use a learning algorithm to predict tomorrow's temperature (in degrees Centigrade/Fahrenheit)
 - **Problem 2:** Given historical data of two football teams' wins/losses, examine the statistics of the two teams and predict which team will win tomorrow's match.
 - Exercise 2: Turn the following regression problem into a classification problem
 - Given sizes (and energy labels, ages, distance to city center) of houses, predict their price.

Unsupervised learning

- Train on unlabeled data to look for meaningful connection
 - Approach problems with little or no idea what our results should look like
 - Often a necessary step in better understanding a dataset before attempting to solve a supervised-learning problem
- Good at
 - Clustering: Splitting the dataset into groups based on similarity, without knowing what each group represents
 - Take a collection of 1M different genes and group these genes into groups that are somehow similar or related by different variables, such as lifespan, location, roles.
 - Anomaly detection: identifying rare items, events or observations
 - Automatic video surveillance for theft detection in ATM machines

18

Clustering vs classification



Clustering or classification?

19

Semi-supervised learning

- Mix of supervised and unsupervised learning
 - Training data might be provided, but the model is free to explore the data on its own and develop its own understanding of the dataset
 - Why: performance usually improves when trained on labeled datasets, but labeling data can be time consuming and expensive
 - Strikes a middle ground between the performance of supervised learning and the efficiency of unsupervised learning
- Good at
 - Machine translation: teaching algorithms to translate language based on less than a full dictionary of words
 - Fraud detection: identifying cases of fraud when you only have a few examples
 - Labelling data: algorithms trained on small data sets can learn to apply data labels to larger sets automatically

20

Reinforcement learning

- Teach a machine to complete a multi-step process with defined rules
 - Positive or negative cues are given
 - The algorithm decides on its own what steps to take to maximize reward
- Good at
 - Robotics: robots can learn to perform tasks
 - Video gameplay: to teach bots to play a number of video games
 - Example: DeepMind's AlphaGo
 - Resource management: Given finite resources and a defined goal, help enterprises plan out how to allocate resources
- Mostly a research area and no significant successes beyond games

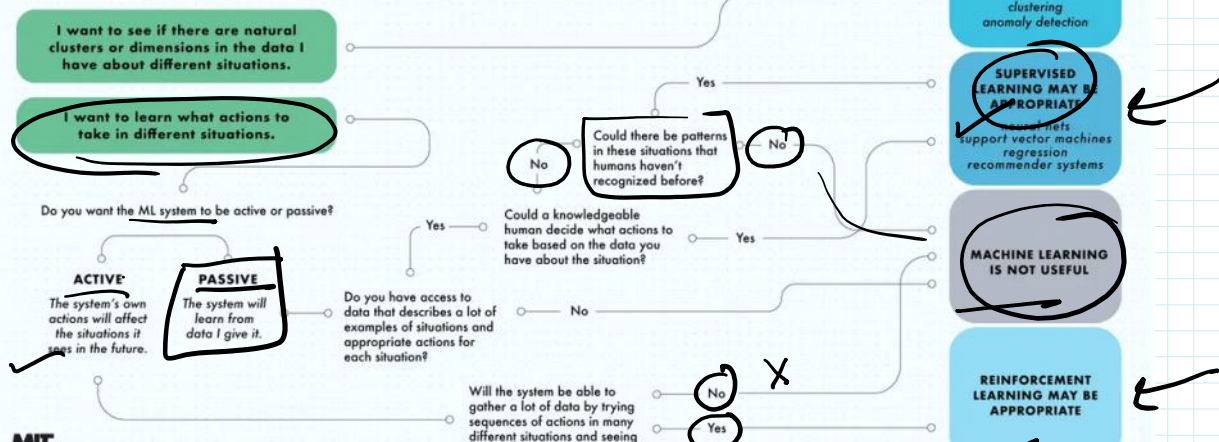
21

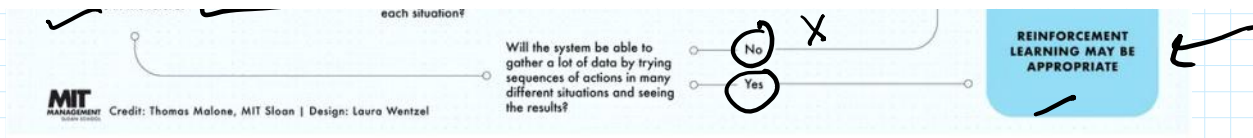
The advantages of using machine learning

- Successful fields (near-human-level)
 - Image classification
 - Speech recognition
 - Handwriting transcription
 - Autonomous driving
- Improvement in many tasks
 - Machine translation
 - Text-to-speech conversion
 - Ad targeting
 - Search on the web
 - ...

22

What do you want the machine learning system to do?





Source: Thomas Malone | MIT Sloan. See: <https://bit.ly/3gyRho2>. Figure 2.

Forms of Learning

The field of machine learning usually distinguishes four forms of learning – supervised learning, unsupervised learning, reinforcement learning and learning based natural process.

1. Supervised/ Direct Learning: **Classification and Regression**
2. Unsupervised Learning: **Cluster analysis and Association analysis**
3. Reinforcement Learning

Supervised Learning

Supervised learning: In supervised learning, the algorithm is provided with a dataset consisting of input-output pairs, and it learns to predict the output labels for new, unseen input data. **Regression and Classification**

Supervised Learning: Regression

▪Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed.

▪It predicts continuous/real values such as **temperature, age, salary, price, etc.**

▪**House price prediction:** Predicting the selling price of a house based on features such as size, location, and number of bedrooms.

▪**Stock price prediction:** Predicting the future price of a stock based on historical trading data and other relevant factors.

Advertisement	Sales
\$90	\$1000
\$120	\$1300
\$150	\$1800
\$100	\$1200
\$130	\$1380
\$200	??

26

Supervised Learning: Classification

Classification: Supervised learning algorithms can be used for classification tasks, where the goal is to categorize input data into predefined classes or categories. For example:

Email spam detection: Classifying emails as spam or non-spam based on their content and characteristics.

Sentiment analysis: Classifying text documents (e.g., reviews, social media posts) as positive, negative, or neutral based on their sentiment.

Cancer diagnosis: Predicting whether a patient has cancer based on medical imaging data (e.g., mammograms, MRIs) and clinical features.



Source: javapoint.com

Unsupervised Learning: Clustering

Unsupervised learning is a type of machine learning where the algorithm **learns patterns and structures from input data** without explicit supervision or labeled outputs. The algorithm explores the data and identifies hidden patterns or relationships among the input features.

Clustering: Unsupervised learning algorithms can group similar data points together into clusters based on their feature similarities. Examples include:

Customer segmentation: Identifying groups of customers with similar purchasing behavior or preferences for targeted marketing strategies.

Document clustering: Grouping similar documents together based on their content or topics for organization and analysis.

27

Reinforcement Learning

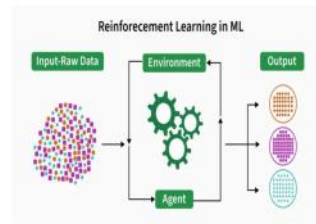
➤ In reinforcement learning, there are no labels. The agent learns by trial and error through interaction with an environment.

➤ An **agent** takes an action

➤ The environment responds with:

- **Reward** (good)
- **Penalty** (bad)

➤ The agent learns a **policy** (what to do in each situation)



Different Learning Types

Criteria	Supervised ML	Unsupervised ML	Reinforcement ML
Definition	Learns by using labelled data	Trained using unlabelled data without any guidance.	Works on interacting with the environment
Type of data	Labelled data	Unlabelled data	No – predefined data
Type of problems	Regression and classification	Association and Clustering	Exploitation or Exploration
Supervision	Extra supervision	No supervision	No supervision
Algorithms	Linear Regression, Logistic Regression, SVM, KNN etc.	K – Means, C – Means, Apriori	Q – Learning, SARSA
Aim	Calculate outcomes	Discover underlying patterns	Learn a series of action
Application	Risk Evaluation, Forecast Sales	Recommendation System, Anomaly Detection	Self Driving Cars, Gaming, Healthcare

Traditional Programming vs Machine Learning

Traditional Programming

- Programmer writes **rules and logic**
- System follows **fixed instructions**
- No learning from data
- Behavior does **not improve automatically**
- Best for **simple and well-defined problems**

Flow:
Input + Rules → Output

Machine Learning

- System learns **patterns from data**
- Rules are **learned automatically**
- Improves performance with **more data**
- Can handle **complex problems**
- Best for **real-world and uncertain tasks**

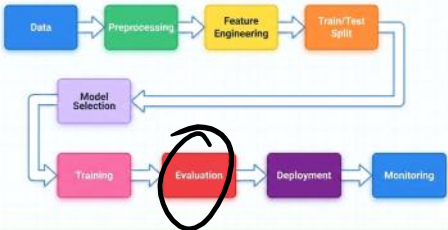
Flow:
Input + Output → Model → Prediction

Traditional Programming vs Machine Learning

Feature	Traditional Programming	Machine Learning
Rule Creation	Manually written	Learned from data
Data Requirement	Low	High
Adaptability	Low	High
Accuracy	Fixed	Improves over time
Human Effort	High coding	High data, less rules
Examples	Calculator, rule engines	Spam filter, face recognition

Machine Learning (ML) pipelining

➤ Machine Learning (ML) pipelining is a structured process that automates and organizes all steps of building an ML model, from raw data to final predictions.



Machine Learning (ML) pipelining

1. Data Collection

➤ Collect data from:

- Databases
- Sensors
- APIs
- Files (CSV, Excel)

➤ Data can be:

- Structured
- Unstructured
- Semi-Structured

➤ Gather relevant and sufficient data

Data Type	Structure	Examples	ML Usage
Structured	Fixed rows & columns	Tables, databases	Direct ML models
Unstructured	No fixed format	Text, images, audio	Feature extraction needed
Semi-Structured	Partial structure	JSON, XML	Preprocessing required

Machine Learning (ML) pipelining

2. Data Preprocessing

- Handling missing values
- Removing duplicates
- Handling outliers
- Encoding categorical data
- Normalization / scaling

Step	What it Does	Simple Example
Handling Missing Values	Fills or removes missing data	Marks: A → 0 or average
Removing Duplicates	Deletes repeated records	Same student ID appears twice
Handling Outliers	Treats extreme values	Salary: 30k, 35k, 500k
Encoding Categorical Data	Converts text to numbers	Male → 1, Female → 0
Normalization / Scaling	Brings values to same range	Height & weight scaled to 0-1

Machine Learning (ML) pipelining

3. Feature Engineering: Good features → better model performance

Step	What it Does	Common Methods / Techniques	Simple Example
➤ Selecting Important Features	Keeps only relevant features	Correlation analysis, Feature importance, Chi-square test	Select Marks, Attendance; remove Name
➤ Creating New Features	Generates new useful features	Feature combination, Domain knowledge, Aggregation	DOB → Age, Height & Weight → BMI
➤ Transforming Features	Changes feature form for better learning	Log transform, Scaling, Binning	Salary → log scale
➤ Dimensionality Reduction	Reduces number of features	PCA, LDA, Autoencoders	100 features → 10 features
➤ Feature Selection Algorithms	Automatically select best features	Forward selection, Backward elimination, LASSO	Select top-k features



Machine Learning (ML) pipelining

4. Data Splitting

➤ Divide dataset into:

- Training set
- Validation set
- Testing set

➤ Common ratio:

- 70% training
- 15% validation
- 15% testing

Machine Learning (ML) pipelining

5. Model Selection

➤ Choose suitable algorithm based on problem:

- Regression
- Classification
- Clustering

➤ Examples:

- Linear Regression
- Decision Trees
- SVM
- Neural Networks

Machine Learning (ML) pipelining

6. Model Training

➤ Train model using training data

➤ Model learns patterns from data

➤ Parameters are adjusted to reduce error

Machine Learning (ML) pipelining

7. Model Evaluation

➤ Test model on unseen data

➤ Use evaluation metrics:

▪ Accuracy ✓

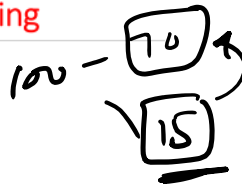
▪ Precision ✓

▪ Recall ✓

▪ F1-score ✓

▪ RMSE ✓

➤ Ensures model generalizes well



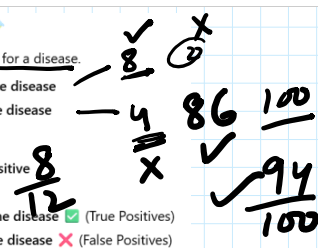
Example: Disease Test

Suppose 100 people are tested for a disease.

- 10 people actually have the disease
- 90 people do not have the disease

Test results:

- Test says 12 people are positive
 - Out of these 12:
 - 8 actually have the disease ✓ (True Positives)
 - 4 do not have the disease ✗ (False Positives)
- Test misses 2 diseased people ✗ (False Negatives)



1 Accuracy

What it means:

➤ How many total predictions are correct?

Formula:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

Here:

$$\text{Correct} = 8 \text{ (TP)} + 86 \text{ (TN)} = 94$$

$$\text{Total} = 100$$

$$\text{Accuracy} = 94\%$$

2 Precision

What it means:

➤ Out of all predicted positives, how many are actually positive?

Formula:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Here:

$$\frac{8}{12} = 66.7\%$$

- High precision = fewer false alarms
- Important when false positives are costly

3 Recall (Sensitivity)

What it means:

➤ Out of all actual positives, how many did we correctly find?

Formula:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Here:

$$\frac{8}{10} = 80\%$$

- High recall = fewer missed cases
- Important when missing a positive is dangerous (e.g., disease, fraud)

HM

80%
M

Machine Learning (ML) pipelining

8. Hyperparameter Tuning

➤ It is the process of choosing *the best values for parameters that are set before training* a machine learning model.

➤ Improve model performance

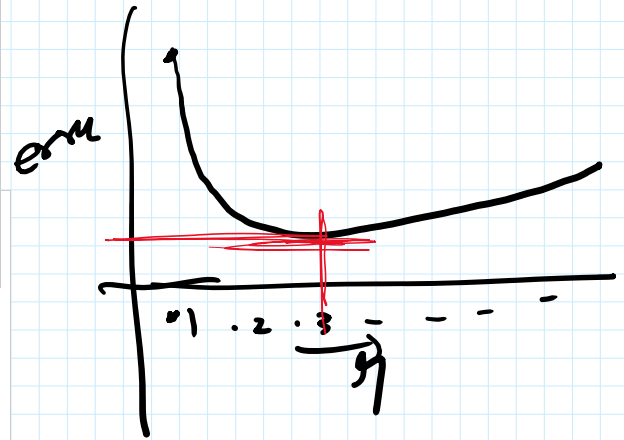
➤ Techniques:

- Grid Search: Expensive, tries all possible combinations of hyperparameter values
- Random Search: Randomly selects combinations, Often finds good solutions quickly

➤ Examples:

- Learning rate
- Number of layers
- Tree depth

Model	Key Hyperparameters
Decision Tree	Max depth, Min samples, Criterion
ANN	Learning rate, Layers, Neurons, Optimizer
Linear Regression	Learning rate, Regularization



Machine Learning (ML) pipelining

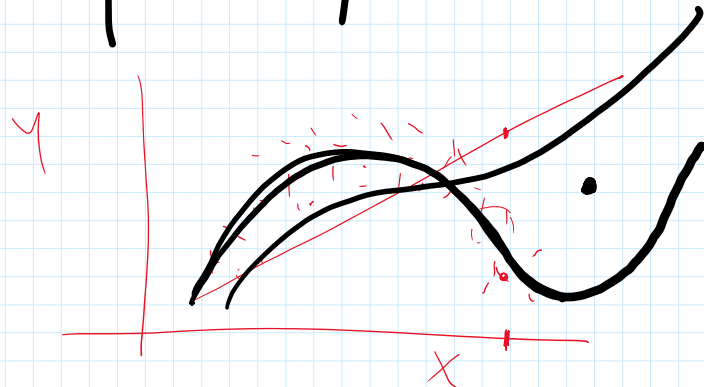
9. Model Validation

➤ Check for:

- Overfitting
- Underfitting

➤ **Adjust model if needed**

Aspect	Overfitting	Underfitting
➤ Model behavior	Too complex	Too simple
➤ Training accuracy	High	Low
➤ Test accuracy	Low	Low
➤ Main issue	Memorization	No learning
➤ Solution	Simplify model	Increase complexity



Machine Learning (ML) pipelining

10. Model Deployment

➤ Deploy model into real-world system

➤ Methods:

- ~~Web API~~
- Cloud service
- Mobile or edge devices

Reg.
/ LASSO

Machine Learning (ML) pipelining

11. Monitoring & Maintenance

- Monitor performance over time
- Detect: Data drift or Concept drift
 - ✓ Data drift: *The model sees different kind of data than it was trained on.*
 - **Example of Data Drift:**
 - Training data: Age: 18–30 Income: 20k–40k
 - New data: Age: 40–60 Income: 60k–100k
 - The problem is same, but data values have shifted.
 - ✓ Concept drift: *The meaning of the target changes i.e. Relationship between input and output changes*
 - Spam detection:
 - Old rules → spam keywords
 - New spams use different patterns
- Retrain model if required

24/10/20

14

Data Preprocessing

- Data preprocessing is the process of cleaning, transforming, and preparing raw data so that it can be used effectively by machine learning models.
- Why it is needed?
 - Real-world data is incomplete, noisy, and inconsistent
 - ✓ ML models cannot learn properly from raw data
 - Improves accuracy, speed, and reliability

Good data → Good model ✓
Bad data → Bad model

24/10/20

15

Data Preprocessing

Steps in Data Preprocessing

- Handling missing values
- Removing duplicates
- Handling outliers
- Encoding categorical data
- Normalization / Scaling

24/10/20

16

Data Preprocessing: Missing Values

Handling Missing Values

- Missing values occur when some data is not recorded.
- Most ML model perform mathematical operations (sum, mean, variance, distance, matrix multiplication, etc.) during training and missing values like (Nan, Null, Empty cells) can not participate.
- Missing data leads to errors, biased learning, incorrect patterns, and reduced model performance.

School ID	Name	Address	City	Subject	Score	Rank	Grade
101.0	A	123 Main St	Mumbai	Math	85.0	2	B
102.0	B	456 Oak Ave	Delhi	English	92.0	1	A
103.0	C	789 Pine Ln	Bangalore	Science	78.0	4	C
104.0	D	101 Elm St	Chennai	Math	89.0	3	B
105.0	E	NaN	Kolkata	History	NaN	8	D
106.0	F	222 Maple Rd.	NaN	Math	55.0	1	A
107.0	G	444 Cedar Blvd	Pune	Science	80.0	5	C
108.0	H	555 Birch Dr	Jalpur	English	88.0	3	B

24/03/24

17

Data Preprocessing: Missing Values

Reasons Behind Missing Values in the Dataset

- Technical issues: Failed data collection or errors during data transmission.
- Human errors: Mistakes like incorrect data entry or oversights during data processing.
- Privacy concerns: Missing sensitive or personal information due to confidentiality policies.
- Data processing issues: Errors that occur during data preparation.

Types of Missing Values

- MCAR (Missing Completely at Random): Missingness occurs randomly and is not related to any variable in the dataset.
- MAR (Missing at Random): Missingness depends on other observed variables, not on the missing value itself.
- MNAR (Missing Not at Random): Missingness is directly related to the value that is missing (e.g., high-income individuals not reporting income).



24/03/24

18

Data Preprocessing: Missing Values

MCAR → Safe to delete
MAR → Delete causes bias
MNAR → Deletion is dangerous

Importance of MCAR, MAR, MNAR

Aspect	MCAR (Missing Completely At Random)	MAR (Missing At Random)	MNAR (Missing Not At Random)
Definition	Missingness unrelated to any variable	Missingness depends on observed variables	Missingness depends on missing value itself
Root Cause	Random failures, accidental skips	Systematic but explainable	Intentional or value-dependent
Dependency	None	Other features	The missing feature itself
Typical Scenario	Sensor randomly off	Income missing for students	High income hidden
Can Model Learn Pattern?	No pattern exists	Partial pattern exists	Hidden pattern
Safe to Remove Rows?	Yes	No	Never
Effect of Row Deletion	Only data size reduces	Biased dataset	Removes critical population
Bias Introduced	Low	Medium	Very High
Impact on Generalization	Minimal	Poor	Dangerous
Best Handling Strategy	Drop rows or simple impute	Conditional / model-based imputation	Domain rules + missing indicator Domain rules + missing indicator

24/03/24

19

Data Preprocessing: Missing Values

Handling Missing values

```
import pandas as pd
import numpy as np
```

```
data = {
    'School ID': [101, 102, 103, np.nan, 105, 106, 107, 108],
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'],
    'Address': ['123 Main St', '456 Oak Ave', '789 Pine Ln', '101 Elm St', np.nan, '222 Maple Rd', '444 Cedar Blvd', '555 Birch Dr'],
    'City': ['Mumbai', 'Delhi', 'Bengaluru', 'Chennai', 'Kolkata', np.nan, 'Pune', 'Jaipur'],
    'Subject': ['Math', 'English', 'Science', 'Math', 'History', 'Math', 'Science', 'English'],
    'Marks': [85, 92, 78, 89, np.nan, 95, 80, 88],
    'Rank': [2, 1, 4, 3, 8, 1, 5, 3],
    'Grade': ['B', 'A', 'C', 'B', 'D', 'A', 'C', 'B']
}
```

```
Sample DataFrame:
School ID Name Address City Subject Marks Rank Grade
0 101.0 A 123 Main St Mumbai Math 85.0 2 B
1 102.0 B 456 Oak Ave Delhi English 92.0 1 A
2 103.0 C 789 Pine Ln Bengaluru Science 78.0 4 C
3 NaN D 101 Elm St Chennai Math 89.0 3 B
4 105.0 E NaN Kolkata History NaN 8 D
5 106.0 F 222 Maple Rd Pune Math 95.0 1 A
6 107.0 G 444 Cedar Blvd Jaipur Science 80.0 5 C
7 108.0 H 555 Birch Dr Jaipur English 88.0 3 B
```

```
df = pd.DataFrame(data)
print("Sample DataFrame:")
print(df)
```

24/3/20

20

Data Preprocessing: Missing Values

Removing Rows with Missing Values

```
df_cleaned = df.dropna()
print("\nDataFrame after removing rows with missing values:")
print(df_cleaned)
```

```
DataFrame after removing rows with missing values:
School ID Name Address City Subject Marks Rank Grade
0 101.0 A 123 Main St Mumbai Math 85.0 2 B
1 102.0 B 456 Oak Ave Delhi English 92.0 1 A
2 103.0 C 789 Pine Ln Bengaluru Science 78.0 4 C
6 107.0 G 444 Cedar Blvd Pune Science 80.0 5 C
7 108.0 H 555 Birch Dr Jaipur English 88.0 3 B
```

Row vs Column Removal – Quick Decision Table

Situation	Remove Row	Remove Column
< 5% missing	✓ Yes	✗ No
> 50% missing	✗ No	✓ Yes
Target missing	✓ Yes	✗ No
Small dataset	✗ Avoid	✗ Avoid
Feature unimportant	✗ No	✓ Yes
MCAR	✓ Yes	⚠ Depends
MNAR	✗ No	✗ No

24/3/20

21

Data Preprocessing: Missing Values

Imputation Methods

Mean, Median and Mode Imputation: This method involves replacing missing values with the mean, median or mode of the relevant variable. It's a simple approach but it doesn't account for the relationships between variables.

```
mean_imputation = df['Marks'].fillna(df['Marks'].mean())
median_imputation = df['Marks'].fillna(df['Marks'].median())
mode_imputation = df['Marks'].fillna(df['Marks'].mode()[0])

print("\nImputation using Mean:")
print(mean_imputation)
0 85.0
1 92.0
2 78.0
3 89.0
4 86.714286
5 95.000000
6 80.000000
7 88.000000
Name: Marks, dtype: float64

print("\nImputation using Median:")
print(median_imputation)
0 85.0
1 92.0
2 78.0
3 89.0
4 78.0
5 95.0
6 80.0
7 88.0
Name: Marks, dtype: float64

print("\nImputation using Mode:")
print(mode_imputation)
0 85.0
1 92.0
2 78.0
3 89.0
4 78.0
5 95.0
6 80.0
7 88.0
Name: Marks, dtype: float64
```

24/3/20

22

Data Preprocessing: Missing Values

Forward and Backward Fill :

These methods fill missing values using the closest available values in the column. They are useful for ordered or time-series data.

```
forward_fill = df['Marks'].fillna(method='ffill')
backward_fill = df['Marks'].fillna(method='bfill')
```

```
print("\nForward Fill:")
print(forward_fill)
```

```
print("\nBackward Fill:")
print(backward_fill)
```

In forward fill, if the first value is missing, it remains unfilled because no previous value exists. Similarly, in backward fill, if the last value is missing, it remains unfilled because no next value exists. These methods rely on neighboring values and therefore fail at dataset boundaries

```
0  85.0
1  92.0
2  78.0
3  89.0
4  89.0 ← filled value
5  95.0
6  80.0
7  88.0
Name: Marks, dtype: float64

0  85.0
1  92.0
2  78.0
3  89.0
4  95.0 ← filled value
5  95.0
6  80.0
7  88.0
Name: Marks, dtype: float64
```

24/3/20

23

Data Preprocessing: Handling Duplication

➤ They occur when identical rows or entries appear multiple times in a dataset.

➤ Duplicates could happen due to: **Data entry errors** or **Merging datasets** ✓

➤ Why Duplicates Can Cause Problems?

• **Skewed Analysis:** Misleading conclusions such as an wrong average salary.

• **Inaccurate Models:** It can cause machine learning models to overfit.

• **Increased Computational Costs & Data Redundancy and Complexity**

24/3/20

24

Data Preprocessing: Handling Duplication

➤ Identifying Duplicates

import pandas as pd

```
data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'David'],
    'Age': [25, 30, 25, 35, 30, 40],
    'City': ['New York', 'Los Angeles', 'New York', 'Chicago', 'Los Angeles', 'San Francisco']
}
df = pd.DataFrame(data)
df
```

```
duplicates = df.duplicated()
```

```
duplicates
```

	Name	Age	City	
0	Alice	25	New York	
1	Bob	30	Los Angeles	
2	Alice	25	New York	0
3	Charlie	35	Chicago	1
4	Bob	30	Los Angeles	1
5	David	40	San Francisco	2

dtype: bool

24/3/20

25

Data Preprocessing: Handling Duplication

➤ Removing Duplicates

Based on Specific Columns

```
df_no_duplicates_columns = df.drop_duplicates(subset=['Name', 'City'])  
(df_no_duplicates_columns)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
3	Charlie	35	Chicago
5	David	40	San Francisco

Keeping the First or Last Occurrence

```
df_keep_last = df.drop_duplicates(keep='last')  
(df_keep_last)
```

	Name	Age	City
2	Alice	25	New York
3	Charlie	35	Chicago
4	Bob	30	Los Angeles
5	David	40	San Francisco

24/03/20

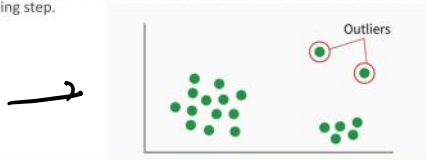
26

Data Preprocessing: Outlier Detection

➤ Outliers are data points that deviate significantly from the general distribution of the dataset.

➤ They may occur due to errors in data collection, natural variation or rare events.

➤ While sometimes they contain useful insights like in fraud detection but in many cases they negatively affect model accuracy and skew results making outlier detection a crucial preprocessing step.



24/03/20

27

Data Preprocessing: Outlier Detection

➤ **Impact on ML models:** Can bias parameter estimation, increase variance and reduce model accuracy.

➤ **Sources:** Data entry errors, measurement noise, genuine rare events.

➤ **Handling methods:** Removal, transformation or robust algorithms less sensitive to outliers.

➤ **Domain dependence:** What is considered an outlier in one domain (e.g., finance) may be normal in another.

Types of Outliers

➤ **Global Outliers (Point Anomalies):** Individual data points that lie far from the rest.

✓ **Contextual Outliers:** Outliers relative to a specific context or condition. Example: 30°C might be normal in summer but an outlier in winter.

➤ **Collective Outliers:** A group of related data points behaving anomalously together.

24/03/20

28

Data Preprocessing: Outlier Detection

Outliers Detection Methods

Step 1: Import Libraries and Load Dataset

Here we will import `numpy`, `pandas`, `matplotlib`, `seaborn`, `sklearn` and `scipy`.

Dataset used: `winequality-red.csv`

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from scipy import stats

df = pd.read_csv("winequality-red.csv")
print(df.shape)
print(df.head())
data = df.drop("quality", axis=1)
```

	volatile acidity	citric acid	residual sugar	chlorides	
0	7.2	0.70	0.00	1.9	0.000
1	7.8	0.00	0.00	2.6	0.000
2	7.4	0.70	0.00	2.3	0.000
3	11.2	0.20	0.50	1.9	0.000
4	7.4	0.70	0.00	1.9	0.000

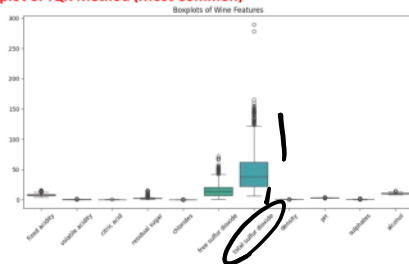
	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	
0	11.0	34.0	0.9978	3.51	0.50	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9978	3.26	0.65	
3	17.0	69.0	0.9980	3.30	0.58	
4	11.0	34.0	0.9978	3.51	0.50	

	alcohol	quality
0	9.4	5
1	9.8	5
2	9.8	5
3	9.8	6
4	9.4	5

Data Preprocessing: Outlier Detection

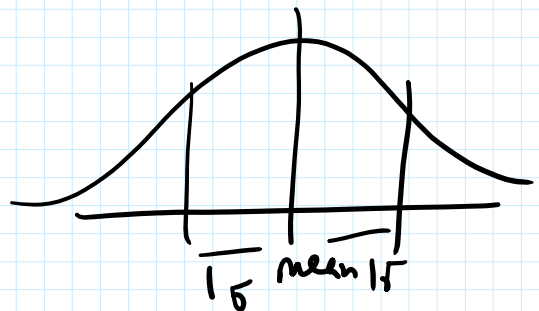
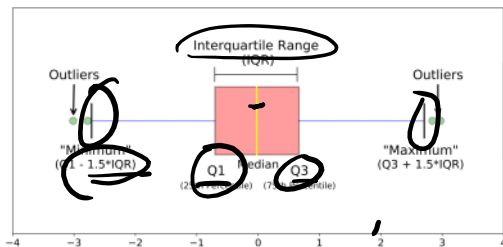
Outliers Detection Methods: Boxplot or IQR Method (Most Common)

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=data)
plt.xticks(rotation=45)
plt.title("Boxplots of Wine Features")
plt.show()
```



Data Preprocessing: Outlier Detection

What is Boxplot?



Data Preprocessing: Outlier Detection

IQR Method (Most Common) to have data without outlier

```
# Calculate quartiles
Q1 = df["total sulfur dioxide"].quantile(0.25)
Q3 = df["total sulfur dioxide"].quantile(0.75)
print("First quartile = ", Q1)
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	
0	7.2	0.70	0.00	1.9	0.000	
1	7.8	0.00	0.00	2.6	0.000	
2	7.4	0.70	0.00	2.3	0.000	
3	11.2	0.20	0.50	1.9	0.000	
4	7.4	0.70	0.00	1.9	0.000	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	
0	11.0	34.0	0.9978	3.51	0.50	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9978	3.26	0.65	
3	17.0	69.0	0.9980	3.30	0.58	
4	11.0	34.0	0.9978	3.51	0.50	

	alcohol	quality
0	9.4	5
1	9.8	5
2	9.8	5
3	9.8	6
4	9.4	5

IQR Method (Most Common) to have data without outlier

```
# Calculate quartiles
df["total sulfur dioxide"] = df["total sulfur dioxide"].quantile(0.25)
Q3 = df["total sulfur dioxide"].quantile(0.75)
print("First quartile =", Q1)
print("Third quartile =", Q3)
# Interquartile Range
IQR = Q3 - Q1
print(IQR)

# Outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter outliers
df_no_outliers = df[
    (df["total sulfur dioxide"] >= lower_bound) &
    (df["total sulfur dioxide"] <= upper_bound)
]
print(df_no_outliers)
```

```
First quartile = 22.8
Third quartile = 32.8
IQR = 10.0

# Filter outliers
df_no_outliers = df[
    (df["total sulfur dioxide"] >= lower_bound) &
    (df["total sulfur dioxide"] <= upper_bound)
]
print(df_no_outliers)
```

[1000 rows x 12 columns]

Data Preprocessing: Outlier Detection

Z-Score Method

Z-Score method is a statistical technique that detects outliers based on how far a data point is from the mean, measured in terms of standard deviations.

It assumes the data follow a normal distribution.

A point with a very high or low Z-score (typically $|Z| > 3$) is flagged as an outlier because it lies in the extreme tails of the distribution.

$$Z = \frac{x - \mu}{\sigma}$$

Z: The Z-score (standard score): It tells us how many standard deviations a data point is away from the mean.

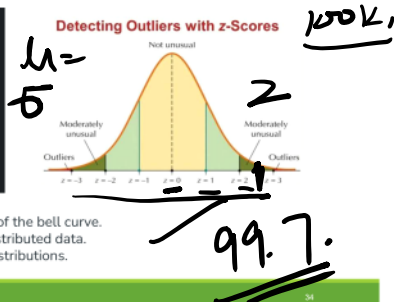
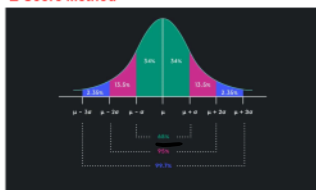
x: The actual data value (the observation we are testing).

μ : The mean of the dataset (average of all data points).

σ : The standard deviation of the dataset (a measure of spread/variability).

Data Preprocessing: Outlier Detection

Z-Score Method



- Intuition: Outliers are "too far" from the center of the bell curve.
- Pros: Simple, fast, works well with normally distributed data.
- Cons: Not reliable for skewed or non-normal distributions.

Data Preprocessing: Outlier Detection

Z-Score Method

```
z_scores = np.abs(stats.zscore(data))
outliers_z = np.where(z_scores > 3)
```

```
print("Outlier positions (row, col):")
print(list(zip(outliers_z[0][:10], outliers_z[1][:10])))
```

- `stats.zscore(data)` computes Z-scores for all columns.
- `np.where(z_scores > 3)` finds data points with $|Z| > 3$.
- We print the first few row-column indices where outliers occur.

Outlier positions (row, col):
[(np.int64(13), np.int64(9)), (np.int64(14), np.int64(5)), (np.int64(15), np.int64(5)), ...,
(np.int64(42), np.int64(4))]

2/4/2020

35

Data Preprocessing: Outlier Detection

Local Outlier Factor (LOF)

- The Local Outlier Factor (LOF) method is a density-based anomaly detection technique that compares the local density of a data point to that of its neighbors. If a point has significantly lower density than its neighbors, it is flagged as an outlier.

How it works:

- For each point, find k-nearest neighbors.
- Estimate local density based on neighbor distances.
- Compare the density of the point with its neighbors.
- If the point's density $\ll$ neighbors $\rightarrow$ outlier.

Pros: Works well with clusters of varying density.

Cons: Sensitive to choice of k (neighbors).

2/4/2020

36

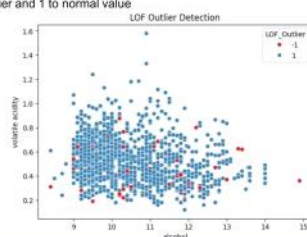
Data Preprocessing: Outlier Detection

Local Outlier Factor (LOF)

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
y_pred_lof = lof.fit_predict(data) # finding assigning -1 to outlier and 1 to normal value
```

```
df["LOF_Outlier"] = y_pred_lof # adding new column
print(df["LOF_Outlier"].value_counts()) # counting -1 and 1
```

```
plt.figure(figsize=(7, 5)) # 7 inches by 5 inches
sns.scatterplot(x="alcohol", y="volatile acidity",
               data=df, hue="LOF_Outlier", palette="Set1")
plt.title("LOF Outlier Detection")
plt.show()
```



2/4/2020

37

Data Preprocessing: Outlier Detection

Technique	Type	Key Idea	Works Well For	Pros	Cons
Z-Score	Statistical	Flags data points that lie far from the mean in terms of standard deviation	Normally distributed, continuous numerical data	Simple, fast, easy to implement	Not reliable for skewed or non-normal data; sensitive to extreme values
IQR (Interquartile Range)	Statistical	Flags values outside the range $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$	Univariate data, box-plot based analysis	Robust to extreme values; non-parametric	Less effective for highly skewed or multimodal distributions
Local Outlier Factor (LOF)	Density-based	Compares local density of a point with densities of its neighbors	Data with clusters or varying densities	Excellent at detecting local outliers	Sensitive to number of neighbors (k); computationally expensive

24/10/20

38

Data Preprocessing: Categorical Data Encoding

- Categorical data represents variables with distinct categories such as labels, names, or types.
- Machine learning models require numerical inputs, so categorical data must be converted into numeric form.
- Proper encoding enables better model understanding, improving prediction accuracy and reducing bias.

Types of Categorical Data

1. Nominal Data: Nominal data consists of categories without any inherent order or ranking. These are simple labels used to classify data.

•Example: 'Red', 'Blue', 'Green' (Car Color).

2. Ordinal Data: Ordinal data includes categories with a defined order or ranking, where the relationship between values is important.

•Example: 'Low', 'Medium', 'High' (Car Engine Power).

24/10/20

39

Data Preprocessing: Categorical Data Encoding

Categorical Data Encoding Techniques

- One-Hot Encoding** : Converts categories into binary columns.
- Label Encoding** : Assigns a unique integer to each category.
- Ordinal Encoding** : Maps categories to ordered integers.
- Binary Encoding** : Converts categories into binary digits.
- Count (Frequency) Encoding** : Replaces categories with their frequency or count.
- Target Encoding** : Replaces categories with the mean target value for each category.

24/10/20

40

Data Preprocessing: Categorical Data Encoding

Label Encoding: assigns each category a unique integer.

➤ Used in tree-based models like Decision Trees or XGBoost.

➤ **Pros:** Simple and memory-efficient.

➤ **Cons:** Introduces implicit order which may be misinterpreted by non-tree models when used with nominal data.

Color	Color
Red	0
Green	1
Blue	2
Red	0

```
from sklearn.preprocessing import LabelEncoder
data = ['Red', 'Green', 'Blue', 'Red']
le = LabelEncoder()
encoded_data = le.fit_transform(data)
print("Encoded Data: {encoded_data}")
```

Encoded Data: [0 1 2 0]

24/03/20

41

Data Preprocessing: Categorical Data Encoding

One-Hot Encoding: converts categories into binary columns with each column representing one category.

➤ It prevents false ordering but can lead to high dimensionality if there are many unique values.

➤ Used in linear models, logistic regression and neural networks.

➤ **Pros:** Does not assume order; widely supported.

➤ **Cons:** Can cause high dimensionality and sparse data when feature has many categories.

Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1
Green	0	1	0

```
import pandas as pd
data = ['Red', 'Blue', 'Green', 'Red']
df = pd.DataFrame(data, columns=['Color'])
one_hot_encoded = pd.get_dummies(df['Color'])
print(one_hot_encoded)
```

```
   Blue  Green  Red
0  False  False  True
1   True  False  False
2  False   True  False
3  False  False  True
```

24/03/20

42

Data Preprocessing: Categorical Data Encoding

Ordinal Encoding: maps categories to integers while preserving their natural order.

➤ This works well for ordered data like ratings but is not suitable for nominal variables.

➤ Used for ordered features like ratings or education levels.

➤ **Pros:** Maintains order; reduces dimensionality.

➤ **Cons:** Not suitable for nominal categories.

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

```
from sklearn.preprocessing import OrdinalEncoder
data = [['Low'], ['Medium'], ['High'], ['Medium'], ['Low']]
encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
encoded_data = encoder.fit_transform(data)
print("Encoded Ordinal Data: {encoded_data}")
```

```
Encoded Ordinal Data: [[0.]
 [1.]
 [2.]
 [1.]
 [0.]]
```

24/03/20

43

Data Preprocessing: Categorical Data Encoding

Ordinal Encoding – When, Where, and Why?

Ordinal encoding **preserves order, not true distance**
 Models that rely mainly on **ordering or thresholds** → work well
 Models that rely **heavily on distance or magnitude** → fail

ML Model / Technique	Suitable for Ordinal Encoding?	When It Is Useful	Why It Works / Fails
Linear Regression	Yes (with care)	When categories have a true order and roughly linear effect	Uses ordinal values to learn monotonic trends; exact distance is less critical
Logistic Regression	Yes	Ordered categories affecting probability (risk level, rating)	Learns monotonic relationship between ordinal feature and log-odds
Decision Tree	Yes (Very suitable)	Any ordered categorical feature	Tree splits use thresholds; ordering helps meaningful splits
Random Forest	Yes	Ordered categories with mixed features	Ensemble of trees benefits from order; robust to spacing issues
Gradient Boosting (XGBoost, LightGBM)	Yes	High-cardinality ordered features	Efficient splits; preserves order with fewer features
K-Means Clustering	Conditionally Yes	Only when order AND spacing are meaningful	Distance-based; ordinal values acceptable only if numeric distance reflects similarity
Hierarchical Clustering	Conditionally Yes	Ordered categories with meaningful distances	Uses distance matrix; artificial spacing can distort clusters
KNN	No	—	Distance-based; ordinal encoding introduces false numeric distances
SVM (especially RBF kernel)	No	—	Margin and kernel depend on true geometric distances
Neural Networks	No	—	Treats values as continuous magnitudes; false spacing is amplified

Data Preprocessing: Categorical Data Encoding

Target Encoding: also known as **Mean Encoding** is a technique where each category in a feature is replaced by the mean of the target variable for that category.

➤ Useful for high-cardinality features like ZIP codes or product IDs.

➤ **Pros:** Captures relationship to target variable.

➤ **Cons:** Risk of overfitting, also must apply smoothing/statistical techniques.

```
# importing libraries
import pandas as pd
```

```
# creating dataset
data={"SubjectName":["s1","s2","s3","s1","s4","s3","s2","s1","s4","s1"],
      "Target":[1,0,1,1,0,0,1,1,0]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

	SubjectName	Target
0	s1	1
1	s2	0
2	s3	1
3	s1	1
4	s4	1
5	s3	0
6	s2	0
7	s1	1
8	s2	1
9	s4	1
10	s1	0

Data Preprocessing: Categorical Data Encoding

```
df.groupby(["SubjectName"])["Target"].count()
```

subjectName	
s1	4
s2	2
s3	2
s4	2

subjectName	
s1	0.750000
s2	0.333333
s3	0.500000
s4	1.000000

```
df.groupby(["SubjectName"])["Target"].mean()
```

```
Mean_encoded_subject =
df.groupby(["SubjectName"])["Target"].mean().to_dict()
df["SubjectName"] =
df["SubjectName"].map(Mean_encoded_subject)
print(df)
```

	SubjectName	Target
0	0.750000	1
1	0.333333	0
2	0.500000	1
3	0.750000	1
4	1.000000	1
5	0.500000	0
6	0.333333	0
7	0.750000	1
8	0.333333	1
9	1.000000	1
10	0.750000	0

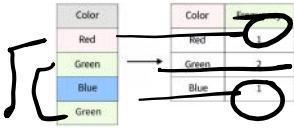
Data Preprocessing: Categorical Data Encoding

Frequency Encoding: assigns categories values based on how often they occur in the dataset.

➤ Effective in retail, e-commerce or clickstream data for popularity trends.

➤ Pros: Low computational and storage requirements.

➤ Cons: Can introduce data leakage if not handled properly.



```
import pandas as pd
data = ["Red", "Green", "Blue", "Red", "Red"]
series_data = pd.Series(data)
frequency_encoding = series_data.value_counts()
```

```
encoded_data = [frequency_encoding[x] for x in data]
print("Encoded Data:", encoded_data)
```

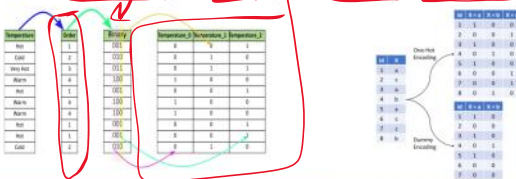
```
Encoded Data: [np.int64(3), np.int64(1), np.int64(1),
np.int64(3), np.int64(3)]
```

Data Preprocessing: Categorical Data Encoding

Binary Encoding: Assigns each category a unique integer, then Converts that integer into its binary representation, and Splits the binary digits into separate columns.

When to Use Binary Encoding: High-cardinality categorical features (hundreds/ thousands of categories), When One-Hot explodes the feature space

➤ With tree-based models (Random Forest, Gradient Boosting) When memory and speed matter



24/05/20

48

Data Preprocessing: Categorical Data Encoding

```
import pandas as pd
import category_encoders as ce
```

```
df = pd.DataFrame({"Color": ["Red", "Blue", "Green", "Yellow"]})
```

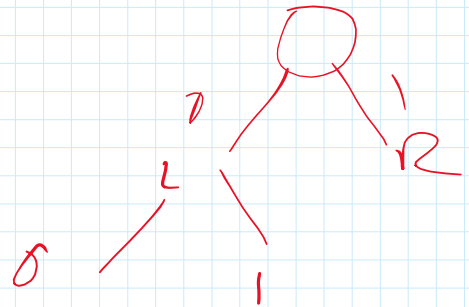
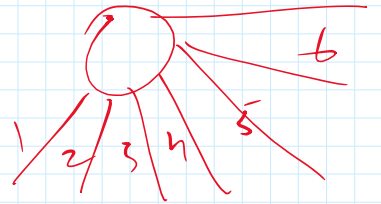
```
encoder = ce.BinaryEncoder(cols=["Color"])
df_bin = encoder.fit_transform(df)
```

```
print(df_bin)
```

```
Color_0 Color_1 Color_2
0 0 0 1
1 0 1 0
2 0 1 1
3 1 0 0
```

24/05/20

49



Data Preprocessing: Categorical Data Encoding

Categorical Data Encoding Techniques – Complete Summary Table

Encoding Technique	Core Idea	Suitable ML Models	Suitable for Clustering?	When to Use
One-Hot Encoding	Creates one binary column per category	Linear Regression, Logistic Regression, SVM, Neural Networks, KNN	Limited (not ideal for K-Means)	Low-cardinality nominal data (no order)
Label Encoding	Assigns integer labels to categories	Decision Tree, Random Forest, Gradient Boosting	Not recommended	Tree-based models with nominal categories
Ordinal Encoding	Encodes categories based on true order	Linear & Logistic Regression, Decision Tree, Random Forest, Gradient Boosting	Conditionally (K-Means, Hierarchical)	Ordered categories (Poor < Good < Excellent)
Binary Encoding	Converts labels to binary digits	Tree-based models, Gradient Boosting, Linear models (large data)	Not suitable	High-cardinality categorical features
Frequency Encoding	Replaces category with its occurrence count	Tree-based models, Boosting models	Sometimes	When frequency itself carries information
Target Encoding	Replaces category with (smoothed) mean of target	Linear Regression, Logistic Regression, Gradient Boosting (XGBoost, LightGBM)	Never	High-cardinality supervised problems

24/03/20

50

Data Preprocessing: Normalization and Scaling

Why scaling / normalization is needed?

➤ Feature scaling is a preprocessing technique used to adjust numerical data so that all features contribute equally during model training.

➤ Features in real data often have different units and ranges:

• Age → 0–100

• Salary → 10,000–1,00,000

• Distance → 0–1

➤ Many ML models:

• compare distances

• use gradients

• assume similar feature scales

➤ Without scaling, large-range features dominate small ones, even if they are less important.

79

Data Preprocessing: Normalization and Scaling

Main Categories of Data Scaling & Normalization

Category	Purpose/ Details
<u>Normalization</u>	- Rescale values to a fixed range (usually 0 to 1) - Keeps relative differences intact and Bring all values in the same measurable range.
<u>Standardization</u>	- Center data around mean with unit variance where mean become 0 Standard deviation becomes 1 - Measure how far values are from the average.
<u>Robust Scaling</u>	- Reduce the effect of outliers - Uses median and interquartile range and Less sensitive to extreme values
<u>Non-linear Scaling</u>	- Reduce skewness in data distribution - Applies log/sqrt/power transformations and Makes data more normal-like - Smooth uneven data distribution.

80

Data Preprocessing: Normalization and Scaling

Data Normalization

Data Preprocessing: Normalization and Scaling

Data Normalization

- Resizes the range of feature values to a specific scale, usually between 0 and 1.
- Normalization ensures that features with different scales or units contribute equally to the model and improves the performance of many machine learning algorithms.
- Maps the minimum and maximum of a feature to a defined range
- Preserves the relative relationships of the original data
- Useful for algorithms that rely on distance metrics such as k-Nearest Neighbours and clustering
- KNN, neural networks, clustering algorithms (like K-Means), and gradient-based models are sensitive to the scale of the input features and can produce better results when the data is normalized.
- Very sensitive to outliers (Extreme values distort the scaling range)

01

	Age	Income
0	20	20000
1	25	40000
2	40	80000
3	60	100000

Data Preprocessing: Normalization and Scaling

Need of Normalization

- When features in a dataset exist on different scales, models may assign more weight to attributes with larger numeric values. Consider the example image below:
- Algorithms that rely on distance, weights, or gradients (e.g., KNN, SVM, Linear Regression) may bias decisions toward high-scale features.

person_name	Salary	Year of experience	Expected Position Level
Aman	100000	10	2
Abhinav	78000	7	4
Ashutosh	32000	5	8
Dishi	55000	6	7
Abhishek	92000	8	3
Avantika	120000	15	1
Ayushi	65750	7	5

The attributes salary and year_of_experience are on different scale and hence attribute salary can take high priority over attribute year_of_experience in the model.

02

Euclidean Distance Calculation (Without Normalization)

$$\begin{aligned}\text{Distance} &= \sqrt{(20 - 25)^2 + (20000 - 40000)^2} \\ &= \sqrt{25 + 400000000} \\ &= \sqrt{400000025} \approx 20000.000625\end{aligned}$$

	Age	Income
0	0.000000	0.000000
1	0.125000	0.250000
2	0.500000	0.750000
3	1.000000	1.000000

Data Preprocessing: Normalization and Scaling

Different Data Normalization Techniques

- 1. Min-Max Normalization/Scaling:** Min-Max scaling is a feature scaling method that rescales data into a fixed range, usually 0 to 1, while preserving the relative differences between values.

General Formula (Custom Range)

$$X_{scaled} = a + \frac{(X - X_{min})(b - a)}{X_{max} - X_{min}}$$

Where:

- X = original value
- Xmin = smallest value in dataset
- Xmax = largest value in dataset
- a = min range either like 0 or -1
- b = max range like 1

03

Data Preprocessing: Normalization and Scaling

Min-Max Normalization/Scaling:

If a=-1

Example

Suppose dataset values are: [10, 20, 30, 40, 50

Here:

- Xmin=10, Xmax=50 a=0 and b=1
- Scale value **30**:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

$$X_{scaled} = \frac{(50-10)/(30-10)}{=20/40 =0.5}$$

Range will be

[0, 0.25, 0.5, 0.75, 1]

$$X_{scaled} = -1 + \frac{(X - X_{min}) \cdot 2}{X_{max} - X_{min}}$$

$$X_{scaled} = -1 + \frac{(30 - 10) \cdot 2}{40}$$

$$= -1 + 1 = 0$$

Range will be

[-1, -0.5, 0, 0.5, 1]

04

Data Preprocessing: Normalization and Scaling

Different Data Normalization Techniques

2. MaxAbs Scaling: MaxAbs scaling rescales values by dividing them by the largest absolute value in the dataset. It keeps the sign of numbers and does not shift the data. After MaxAbs scaling, all values fall within: [-1, 1] range.

General Formula

$$X_{scaled} = \frac{X}{|X_{max}|}$$

Where:

- X = original value
- $|X_{max}|$ = largest absolute value in dataset

05

Data Preprocessing: Normalization and Scaling

Different Data Normalization Techniques : **Min-Max vs MaxAbs Scaling**

Feature	Min-Max Scaling	MaxAbs Scaling
Range	[0, 1] (or custom)	[-1, 1]
Uses	Min & Max values	Maximum absolute value
Data shift	Yes (moves data)	No shift
Zero preserved	Not guaranteed	Always preserved
Outlier sensitivity	High	High

- Choose **Min-Max Scaling** when: You want all values positive, Distance-based models are used and Neural networks expect 0-1 inputs Example uses: KNN, Image processing, Neural networks
- Choose **MaxAbs Scaling** when: Data contains many zeros (sparse data), You want to preserve sign and Memory efficiency matters. Example uses: Text features (TF-IDF) and Sparse matrices

06

Data Preprocessing: Normalization and Scaling

1. Z-Score Standardization: Z-score standardization transforms data so that it is centered around the average and scaled by the spread. It tells how far a value is from the mean.

General Formula

$$Z = \frac{X - \mu}{\sigma}$$

Where:

- X = original value
 - μ = mean (average)
 - σ = standard deviation
- Z-Score Standardization is used when algorithms depend on mean and variance like regression, SVM, PCA, neural networks.
 - Used when data is roughly normal (bell-shaped)
 - Avoid when data has strong outliers, mean and standard deviation get distorted.
 - Avoid when a fixed range is required

Dataset:

[10, 20, 30, 40, 50]

Mean: $\mu=30$

Standard deviation: $\sigma=14.14$

Standardize value **40**:

$Z=(40-30)/14.14 \approx 0.71$

Value 40 is **0.71 standard deviations above the mean.**

87

Data Preprocessing: Normalization and Scaling

➤ **Robust Scaling:** Robust scaling is a feature scaling method that uses median and interquartile range (IQR) instead of mean and standard deviation. It reduces the influence of extreme values.

➤ Some datasets contain outliers like Extremely high or low values, Measurement errors, Rare events, these outliers distort normal scaling methods.

➤ Robust scaling keeps normal data meaningful.

Formula

$$X_{scaled} = \frac{X - \text{Median}}{\text{IQR}}$$

where

$$\text{IQR} = Q_3 - Q_1$$

- Median → center of data
- Q_1 → 25th percentile
- Q_3 → 75th percentile

88

Data Preprocessing: Normalization and Scaling

➤ Dataset with an outlier: [10, 12, 14, 16, 100]

➤ Step 1 — Find statistics Median = 14, $Q_1=12$, $Q_3=16$, $IQR=16-12=4$

➤ Step 2 — Scale value 16 $X_{scaled}=(16-14)/4 = 0.5$

➤ Outlier scaling: $(100-14)/4 = 21.5$

➤ **Normal values are scaled using stable statistics** i.e. typical data is scaled properly

➤ **Outliers remain extreme and outliers do NOT distort the scaling**

➤ In other scaling methods: Outliers stretch the scale and Normal values get compressed

➤ In robust scaling: Normal values keep meaningful spacing and Outliers still appear unusual

➤ Robust scaling = fair scaling for normal data + honest representation of outliers.

89

Data Preprocessing: Normalization and Scaling

- **Log Transformation (Non-Linear Scaling):** Log transformation is a non-linear scaling method that reduces large value gaps and improves data distribution.
- It compresses big numbers more than small ones.
- It reduce right-skewed distribution and compress extreme large values to make data more balanced
- Real-world data often usually contains many small values with few very large values, this creates right skew.
- Log transformation makes the distribution more symmetric.

➤ Formula

$$X' = \log(X + 1)$$

Why +1? It Prevents $\log(0)$, which is undefined.

Example

Income data:
[10, 20, 50, 500, 5000]

After log transform (approx):
[2.4, 3.0, 3.9, 6.2, 8.5]

10

Data Preprocessing: Normalization and Scaling

Choosing the Right Scaling Technique

Technique	When Your Data Looks Like...	What It Fixes	When to Use	Suitable ML Models
Normalization (Min-Max)	Features have very different ranges but no extreme outliers	Brings values into fixed range (0-1) or (-1-1)	When only scale matters	KNN, Neural Networks, Image models, Distance-based algorithms
Standardization (Z-score)	Data is roughly bell-shaped and units differ	Centers data around mean with equal variance	When model depends on distribution	Linear Regression, Logistic Regression, SVM, PCA, Neural Networks
Robust Scaling	Data contains strong outliers or noisy measurements	Protects normal data from outlier distortion	When outliers are present	Regression models, clustering
Non-linear Scaling (Log)	Data is heavily right-skewed with very large values	Reduces skewness and compresses extremes	When distribution is uneven	predictive analytics, neural networks

11

Feature Engineering

Data Preprocessing vs Feature Engineering

Data Preprocessing

Focus: Cleaning and preparing raw data

- Handling missing values
- Removing duplicates
- Handling outliers
- Encoding categorical data
- Normalization / scaling

Goal: Make data usable

Feature Engineering

Focus: Creating meaningful inputs for learning

- Selecting important features
- Creating new features
- Transforming features
- Dimensionality reduction
- Feature selection algorithms
- Goal: Improve model learning & performance

12

Feature Engineering: Selecting Important Features

➤ Feature selection is the process of choosing only the most useful input features. It helps improve model performance, reduces noise and makes results easier to understand.

- Helps remove irrelevant and redundant features
- Improves accuracy and reduces overfitting
- Speeds up model training
- Makes models simpler and easier to interpret

Types of Feature Selection Methods

1. Filter Methods
2. Wrapper methods
3. Embedded methods

13

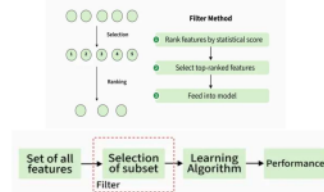
Feature Engineering: Selecting Important Features

1. Filter Methods

Filter methods evaluate each feature independently with target variable. Feature with high correlation with target variable are selected as it means this feature has some relation and can help us in making predictions.

These methods are used to remove irrelevant or redundant features based on statistical tests (correlation) or other criteria.

Hours Studied	Coffee Cups	Exam Score
1	5	40
2	4	50
3	6	60
4	5	70
5	7	80



14

Feature Engineering: Selecting Important Features

Common Filter Methods

➤ **Information Gain:** Measures reduction in entropy when a feature is used. If a feature has very low information gain it can be removed.

➤ **Chi-square test:** Measures dependence between categorical feature and target.

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where: O = observed frequency & E = expected frequency

High χ^2 + small p-value → strong dependency → candidate to keep

Low χ^2 + large p-value → weak evidence → candidate to drop

	Feature	Chi-square	p-value
1	Attendance	4.0	0.045500
2	Gender	4.0	0.045500
0	Study_Method	1.0	0.317311

15

Feature Engineering: Selecting Important Features

Common Filter Methods

- **Pearson's Correlation Coefficient:** Measures linear relationship between feature and target.

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sigma_x \sigma_y}$$

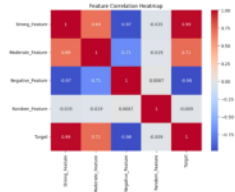
Range:

$$-1 \leq r \leq 1$$

- $|r|$ close to 1 → strong relation → keep

- $r \approx 0$ → weak relation → remove

- A correlation heatmap commonly visualizes Pearson correlation coefficients.



96

Feature Engineering: Selecting Important Features

Advantages and limitations of filter methods

- **Fast and efficient:** Filter methods are computationally inexpensive, making them ideal for large datasets.
- **Easy to implement:** These methods are often built-in to popular machine learning libraries, requiring minimal coding effort.
- **Model Independence:** Filter methods can be used with any type of machine learning model, making them versatile tools.

Limitations

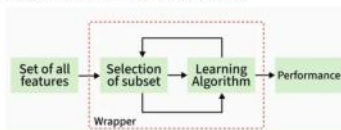
- **Limited interaction with the model:** Since they operate independently, filter methods might miss data interactions that could be important for prediction.
- **Choosing the right metric:** Selecting the appropriate metric for our data and task is important for optimal performance..

97

Feature Engineering: Selecting Important Features

2. Wrapper methods

- Wrapper methods select features by repeatedly training a model using different feature combinations and keeping the ones that improve performance.
- They work like a greedy search: Try a subset of features, Train the model, Measure performance and then Add or remove features based on improvement
- The process continues until a stopping condition is reached, such as: Model performance stops improving or Desired number of features is selected



98

Feature Engineering: Selecting Important Features

Common Wrapper Techniques

Forward Selection: Start with no features and add one at a time based on improvement.

Backward Elimination: Start with all features and remove the least useful ones.

Recursive Feature Elimination (RFE): Removes the least important features step by step.

Example: Forward Selection:

Suppose three features A = Age, B = Salary and C = Attendance with target pass/fail is here.

Step 1 — Test each feature alone: [A] → accuracy = 70%, [B] → accuracy = 60%, [C] → accuracy = 85%

Step 2 — Try combinations with C: [C + A] → 90% [C + B] → 86% select best [C + A] → 90%

Step 3 — Add remaining feature: [C + A + B] → 89% performance drop then stop. Final best subset [C + A] → 90%

99

Feature Engineering: Selecting Important Features

Example: Backward Elimination :

Suppose three features A = Age, B = Salary and C = Attendance with target pass/fail is here.

Step 1 — Start with all features: [A + B + C] → 89%

Step 2 — Remove one feature at a time: remove A → [B + C] → 88%, remove B → [A + C] → 90% (best)

remove C → [A + B] → 72% so keep [A + C] → 90%

Step 3 — remove one more feature: [A] → 70%, [C] → 85% performance drop. Now keep [A + C] → 90%

Example: Recursive Feature Elimination (RFE):

Step1: start with all [A + B + C]

Feature importance ranking: C = high, A = medium, B = low

Remove B and keep A+C best subset.

100

Feature Engineering: Selecting Important Features

Advantages and limitations

➤ **Model-specific optimization:** Wrapper methods directly consider how features influence the model, potentially leading to better performance compared to filter methods.

➤ **Flexible:** These methods can be adapted to various model types and evaluation metrics.

Limitations

➤ **Computationally expensive:** Evaluating different feature combinations can be time-consuming, especially for large datasets.

➤ **Risk of overfitting:** Fine-tuning features to a specific model can lead to an overfitted model that performs poorly on unseen data.

101

Feature Engineering: Selecting Important Features

3. Embedded methods

- Embedded methods perform feature selection during the model training process. They combine the benefits of both filter and wrapper methods.
- The model learns which features are important while it is training. No separate feature selection step. Feature selection is built into the learning process.
- When a model trains, it tries to minimize prediction error. During this process, it naturally learns which features help prediction and which features add noise.
- The model then gives higher importance to useful features and reduces or ignores weak ones

Common Embedded Techniques

- **L1 Regularization (Lasso):** Keeps only features with non-zero coefficients. LASSO automatically shrinks useless feature weights to zero.
- **Decision Trees and Random Forests:** Select features based on impurity reduction.
- **Gradient Boosting:** Pick features that reduce prediction error the most. Each boosting step selects features that reduce prediction error the most.

102

Feature Engineering: Creating New Features

- Creating new features means deriving meaningful attributes from existing data to improve model learning.
- We transform raw data into more informative signals.
- Good features make patterns easier for the model to learn.
- Instead of feeding raw numbers, We create features that represent real-world meaning.
- Transform raw data into meaningful attributes: Height + Weight → BMI
- Combine or group existing features: Study hours × Attendance → Engagement score
- Simplify data for better learning: Birth year → Age group

103

Feature Engineering: Dimensionality Reduction

Dimensionality Reduction?

- The term dimensionality reduction is often reserved for those techniques that reduce the dimensionality of a data set by creating new attributes that are a combination of the old attributes.
- Dimensionality reduction is the process of reducing the number of input features while preserving as much important information as possible.
- It converts high-dimensional data into a lower-dimensional space while preserving important details.
- Purpose:
 - Avoid curse of dimensionality.
 - Reduce amount of time and memory required by data mining algorithms.
 - Allow data to be more easily visualised.
 - May help to eliminate irrelevant features or reduce noise.

104

Feature Engineering: Dimensionality Reduction

The curse of dimensionality?

The curse of dimensionality is the difficulty a model faces when the number of features becomes very large, making it harder to find meaningful patterns and make accurate predictions.

In simpler words:

More features → more confusion → harder learning

106

Feature Engineering: Dimensionality Reduction

Dimensionality Reduction Techniques

1. Feature Selection

Feature selection chooses the most relevant features from the dataset without altering them. It helps remove redundant or irrelevant features, improving model efficiency.

2. Feature Extraction

Feature extraction involves creating new features by combining or transforming the original features. These new features retain most of the dataset's important information in fewer dimensions. Common feature extraction methods are: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), Autoencoders, t-SNE, etc.

The most famous technique here is PCA

106

Feature Engineering: PCA

Principal Component Analysis (PCA)

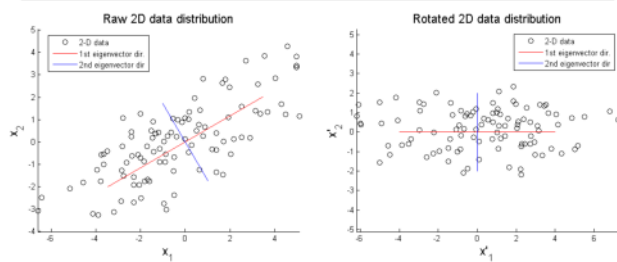
➤ The central idea of principal component analysis (PCA) is to reduce the dimensionality of a data set consisting of a large number of interrelated variables while retaining as much as possible of the variation present in the data set.

➤ This is achieved by transforming to a new set of variables, the principal components (PCs), which are uncorrelated, and which are ordered so that the first few retain most of the variation present in all of the original variables.

➤ Dimensionality reduction removes noise by keeping directions with strong, consistent variation and discarding weak random variation.

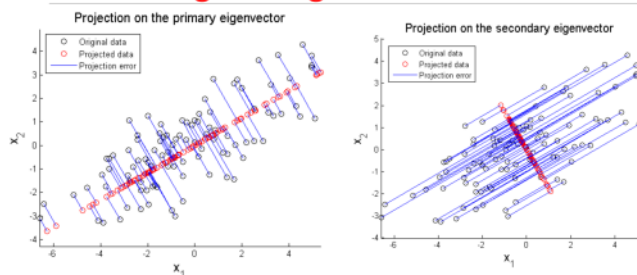
107

Feature Engineering: PCA



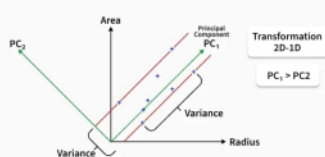
108

Feature Engineering: PCA



109

Feature Engineering: PCA



In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: PC_1 and PC_2 which are the principal components.

- These new axes are rotated versions of the original ones. PC_1 captures the maximum variance in the data meaning it holds the most information while PC_2 captures the remaining variance and is perpendicular to PC_1 .
- The spread of data is much wider along PC_1 than along PC_2 . This is why PC_1 is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC_1 we effectively transform the 2D data into 1D and retain most of the important structure and patterns.

110

Feature Engineering: PCA

Principal Component Analysis (PCA) Steps

Step 1 — Arrange the dataset into a matrix

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots \\ x_{21} & x_{22} & \dots \\ \vdots & \vdots & \dots \end{bmatrix}$$

Step 2 — Standardize (mean center) the data

$$X_{centered} = X - \text{mean}$$

111

Feature Engineering: PCA

Principal Component Analysis (PCA) Steps

Step 3 — Compute covariance matrix to measure how features vary together:

$$C = \frac{1}{n-1} X_{centered}^T X_{centered}$$

Step 4 — Find eigenvalues and eigenvectors

Solve $|C - \lambda I| = 0$

You obtain: Eigenvalues → amount of variance and

Eigenvectors → direction of maximum variance

Step 5 — Sort eigenvalues

➤ Arrange eigenvalues in descending order.

➤ Largest eigenvalue → most important component

112

Feature Engineering: PCA

Principal Component Analysis (PCA) Steps

Step 6 — Select top k eigenvectors

➤ Choose eigenvectors corresponding to largest eigenvalues.

➤ These form the principal components.

Step 7 — Form projection matrix

$$W = [v_1 \ v_2 \ \dots \ v_k]$$

where v_i are selected eigenvectors.

Step 8 — Transform the data

Project original data: $Z = X_{centered} W$

This gives reduced-dimension data.

113

Feature Engineering: PCA

PCA Numerical Example (Find only PC1)

Dataset (2 features): Let the features be: X = Feature 1 and Y = Feature 2

We take 4 points:

Point	X	Y
1	1	1
2	2	3
3	3	2
4	4	5

114

Feature Engineering: PCA

Step 1: Compute mean of X and Y

Mean of X:

$$\bar{X} = \frac{1 + 2 + 3 + 4}{4} = \frac{10}{4} = 2.5$$

Mean of Y:

$$\bar{Y} = \frac{1 + 3 + 2 + 5}{4} = \frac{11}{4} = 2.75$$

Step 2: Center the data (Subtract mean)

Point	X' = X-2.5	Y' = Y-2.75
1	-1.5	-1.75
2	-0.5	0.25
3	0.5	-0.75
4	1.5	2.25

So the centered matrix:

$$X_c = \begin{bmatrix} -1.5 & -1.75 \\ -0.5 & 0.25 \\ 0.5 & -0.75 \\ 1.5 & 2.25 \end{bmatrix}$$

115

Feature Engineering: PCA

Step 3: Compute covariance matrix

Formula:

$$C = \frac{1}{n-1} X_c^T X_c$$

Here n=4, so divide by 3.

Compute variance of Y

Compute variance of X

$$\sum (X')^2 = (-1.5)^2 + (-0.5)^2 + (0.5)^2 + (1.5)^2 \\ = 2.25 + 0.25 + 0.25 + 2.25 = 5$$

$$\text{Var}(X) = \frac{5}{3} = 1.6667$$

$$\sum (Y')^2 = (-1.75)^2 + (0.25)^2 + (-0.75)^2 + (2.25)^2 \\ = 3.0625 + 0.0625 + 0.5625 + 5.0625 = 8.75$$

$$\text{Var}(Y) = \frac{8.75}{3} = 2.9167$$

116

Feature Engineering: PCA

Step 3: Compute covariance XY

$$\sum (X'Y') = (-1.5)(-1.75) + (-0.5)(0.25) + (0.5)(-0.75) + (1.5)(2.25)$$

Now calculate:

- $(-1.5)(-1.75) = 2.625$
- $(-0.5)(0.25) = -0.125$
- $(0.5)(-0.75) = -0.375$
- $(1.5)(2.25) = 3.375$

Sum:

$$2.625 - 0.125 - 0.375 + 3.375 = 5.5$$

$$\text{Cov}(X, Y) = \frac{5.5}{3} = 1.8333$$

Covariance matrix

$$C = \begin{bmatrix} 1.6667 & 1.8333 \\ 1.8333 & 2.9167 \end{bmatrix}$$

117

Feature Engineering: PCA

Step 4: Find eigenvalues (important for PC1)

We solve:

$$|C - \lambda I| = 0$$

$$\begin{vmatrix} 1.6667 - \lambda & 1.8333 \\ 1.8333 & 2.9167 - \lambda \end{vmatrix} = 0$$

Determinant: $(1.6667 - \lambda)(2.9167 - \lambda) - (1.8333)^2 = 0$

$$(4.8612 - 1.6667\lambda - 2.9167\lambda + \lambda^2) - 3.3611 = 0$$

$$\lambda^2 - 4.5834\lambda + (4.8612 - 3.3611) = 0$$

$$\lambda^2 - 4.5834\lambda + 1.5001 = 0$$

Solve quadratic

$$\lambda = \frac{4.5834 \pm \sqrt{(4.5834)^2 - 4(1.5001)}}{2}$$

118

Feature Engineering: PCA

$$(4.5834)^2 \approx 21.006$$

$$4(1.5001) = 6.0004$$

$$\sqrt{21.006 - 6.0004} = \sqrt{15.0056} \approx 3.873$$

Eigenvalue 1:

$$\lambda_1 = \frac{4.5834 + 3.873}{2} = 4.2282$$

Eigenvalue 2:

$$\lambda_2 = \frac{4.5834 - 3.873}{2} = 0.3552$$

119

Feature Engineering: PCA

Step 5: PC1 corresponds to the largest eigenvalue

$$\lambda_1 = 4.2282$$

Now we find eigenvector for λ_1 .
That eigenvector = **PC1 direction**.

Step 6: Find eigenvector for PC1

$$(C - \lambda_1 I)v = 0$$

$$\begin{bmatrix} 1.6667 - 4.2282 & 1.8333 \\ 1.8333 & 2.9167 - 4.2282 \end{bmatrix} = \begin{bmatrix} -2.5615 & 1.8333 \\ 1.8333 & -1.3115 \end{bmatrix}$$

Let eigenvector be:

$$v = \begin{bmatrix} a \\ b \end{bmatrix}$$

Where:

- a = component in X direction
- b = component in Y direction

120

Feature Engineering: PCA

Now multiply with eigenvector [a,b]

$$(C - \lambda_1 I) \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} -2.5615 & 1.8333 \\ 1.8333 & -1.3115 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Rule of matrix multiplication

$$\begin{bmatrix} p & q \\ r & s \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} pa + qb \\ ra + sb \end{bmatrix}$$

First row equation: $(-2.5615)a + (1.8333)b = 0$

Second row equation: $(1.8333)a + (-1.3115)b = 0$

Because both equations are basically the same line (dependent).
That happens because eigenvectors exist only when determinant = 0.
So the system has infinite solutions.
So we can use any one equation.
Let's use first:

$$-2.5615a + 1.8333b = 0$$

121

Feature Engineering: PCA

Now solve for b in terms of a

$$1.8333b = 2.5615a$$

$$b = \frac{2.5615}{1.8333}a$$

$$b \approx 1.397a$$

Eigenvector is not unique.

If $a=1$, then $b=1.397$

So eigenvector can be written as:

$$v = \begin{bmatrix} 1 \\ 1.397 \end{bmatrix}$$

Step 7: Normalize PC1 vector

$$\|v_1\| = \sqrt{1^2 + (1.397)^2} = \sqrt{1 + 1.951} = \sqrt{2.951} = 1.718$$

So normalized PC1:

$$PC1 = \begin{bmatrix} 1/1.718 \\ 1.397/1.718 \end{bmatrix} = \begin{bmatrix} 0.582 \\ 0.813 \end{bmatrix}$$

122

Feature Engineering: PCA

Final Answer: PC1

$$PC1 = \begin{bmatrix} 0.582 \\ 0.813 \end{bmatrix}$$

This means the strongest direction of variance is roughly: $0.582X + 0.813Y$

Step 8: Project data on PC1 (PC1 scores)

$$Z_{PC1} = X_c \cdot PC1$$

Centered points:	$Z_1 = (-1.5)(0.582) + (-1.75)(0.813)$	$Z_4 = (1.5)(0.582) + (2.25)(0.813)$
1.(-1.5, -1.75)	$= -0.873 - 1.423 = -2.296$	$= 0.873 + 1.829 = 2.702$
2.(-0.5, 0.25)		
3.(0.5, -0.75)	$Z_3 = (-0.5)(0.582) + (0.25)(0.813)$	PC1 Scores (new 1D data)
4.(1.5, 2.25)	$= -0.291 + 0.203 = -0.088$	$[-2.296, -0.088, -0.319, 2.702]$
	$Z_3 = (0.5)(0.582) + (-0.75)(0.813)$	So PCA reduced 2D → 1D using PC1.
	$= 0.291 - 0.610 = -0.319$	

123

Feature Engineering: PCA Code

```
import numpy as np
import pandas as pd
# Step 1: Create dataset
data = {
    'X': [1, 2, 3, 4],
    'Y': [1, 3, 2, 5]
}
df = pd.DataFrame(data)
print("Original Data:\n", df)

# Step 2: Convert to matrix
X = df.values
print("X value\n", X)

# Step 3: Mean center
mean = np.mean(X, axis=0)
X_centered = X - mean
print("Mean:", mean)
print("Centered data:\n", X_centered)

# Step 4: Covariance matrix
cov_matrix = np.cov(X_centered.T)
print("\nCovariance matrix:\n",
      cov_matrix)

# Step 5: Eigen decomposition
eigenvalues, eigenvectors =
np.linalg.eig(cov_matrix)
print("\nEigenvalues:\n", eigenvalues)
print("\nEigenvectors:\n", eigenvectors)

# Step 6: Sort eigenvalues
idx = np.argsort(eigenvalues)[::-1]
print("idx\n", idx);
eigenvalues = eigenvalues[idx]
print("eigenvalues\n", eigenvalues);
eigenvectors = eigenvectors[:, idx]
print("eigenvectors\n", eigenvectors);
```

124

Feature Engineering: PCA Code

```
# Select PC1
pcl = eigenvectors[:, 0]
print("PC1\n", pcl)
# Step 7: Force positive direction
if pcl[0] < 0:
    pcl = -pcl

# Normalize (optional - for nice
# comparison)
pcl = pcl / np.linalg.norm(pcl)
print("\nPC1 (matched
direction):", pcl)

# Step 8: Project data
projection = X_centered.dot(pcl)
print("\nProjection onto PC1:\n",
      projection)

--- Original Data:
   X  Y
0  1  1
1  2  3
2  3  2
3  4  5
X value
[[1 1]
 [2 3]
 [3 2]
 [4 5]]
Mean: [2.5  2.75]
Centered data:
[[-1.5 -1.75]
 [-0.5  0.25]
 [ 0.5 -0.75]
 [ 1.5  2.25]]
Covariance matrix:
[[1.46666667 1.83333333]
 [1.83333333 2.91666667]]

Eigenvalues:
[0.35472678 4.22888655]
Eigenvectors:
[[-0.81322627 -0.58194762]
 [ 0.58194762 -0.81322627]]
idx
[1 0]
eigenvalues
[4.22888655 0.35472678]
eigenvectors
[[-0.58194762 -0.81322627]
 [-0.81322627  0.58194762]]
PC1
[-0.58194762 -0.81322627]
PC1 (matched direction): [0.58194762 0.81322627]
Projection onto PC1:
[-2.2966874 -0.08766724 -0.31894589  2.70268854]
```

125

Feature Engineering: PCA Code for 2D and 1D

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# Step 1: Create dataset
data = {
    'Height': [170, 165, 180, 175, 160, 172, 168, 177, 162,
158],
    'Weight': [65, 59, 75, 68, 55, 70, 62, 74, 58, 54],
    'Age': [30, 25, 35, 28, 22, 32, 27, 33, 24, 21],
    'Gender': [1, 0, 1, 1, 0, 1, 0, 1, 0, 0]}
df = pd.DataFrame(data)
print("Original Data:\n", df)

# Step 2: Separate features & target
X = df[['Height', 'Weight', 'Age']]
y = df['Gender']

# Step 3: Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

2103220

DR KP SHARMA, NIT JALANCHAR

126

Feature Engineering: PCA Code for 2D and 1D

```
# PCA -> 2 Dimensions
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_scaled)
df_2d = pd.DataFrame(X_pca_2d, columns=['PC1', 'PC2'])
df_2d['Gender'] = y
print("\nPCA Reduced to 2D:")
print(df_2d)
print("\nVariance captured (2D):")
print(pca_2d.explained_variance_ratio_)

# PCA -> 1 Dimension
pca_1d = PCA(n_components=1)
X_pca_1d = pca_1d.fit_transform(X_scaled)
df_1d = pd.DataFrame(X_pca_1d, columns=['PC1'])
df_1d['Gender'] = y
print("\nPCA Reduced to 1D:")
print(df_1d)
print("\nVariance captured (1D):")
print(pca_1d.explained_variance_ratio_)
```

127

Feature Engineering: PCA Code for 2D and 1D

Original Data:					PCA Reduced to 2D:				PCA Reduced to 1D:		
	Height	Weight	Age	Gender		PC1	PC2	Gender		PC1	Gender
0	170	65	30	1	0	0.479350	0.248005	1	0	0.479350	1
1	165	59	25	0	1	-1.048541	-0.046208	0	1	-1.048541	0
2	180	75	35	1	2	2.737638	0.017043	1	2	2.737638	1
3	175	68	28	1	3	0.874626	-0.506005	1	3	0.874626	1
4	160	55	22	0	4	-2.160684	-0.022405	0	4	-2.160684	0
5	172	70	32	1	5	1.300132	0.336909	1	5	1.300132	1
6	168	62	27	0	6	-0.307430	-0.032200	0	6	-0.307430	0
7	177	74	33	1	7	2.157779	-0.030548	1	7	2.157779	1
8	162	58	24	0	8	-1.501153	0.088028	0	8	-1.501153	0
9	158	54	21	0	9	-2.531716	0.015408	0	9	-2.531716	0

Variance captured (2D):		Variance captured (1D):	
[0.97871061	0.01755646]	[0.97871061]	

128

Evaluation Metrics

17 January 2026 06:58 PM

Performance Metrics

After doing the usual Feature Engineering, Selection, and of course, implementing a model and getting some output in forms of a probability or a class, the next step is to find out how effective is the model based on some metric using test datasets.

The metrics that you choose to evaluate your machine learning model is very important. Choice of metrics influences how the performance of machine learning algorithms is measured and compared.

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- Recall or Sensitivity
- Specificity
- F1 Score
- Area under the curve (AUC)

Confusion Matrix

confusion matrix is one of the most intuitive and easiest metrics used for finding the correctness and accuracy of the model. It is used for Classification problem where the output can be of two or more types of classes.

Let's say we are solving a classification problem where we are predicting whether a person is having cancer or not.

Let's give a label of to our target variable:

1: When a person is having cancer
0: When a person is NOT having cancer.

Alright! Now that we have identified the problem, the confusion matrix, is a table with two dimensions ("Actual" and "Predicted"), and sets of "classes" in both dimensions. Our Actual classifications are columns and Predicted ones are Rows.

Predicted	Actual	
	Positives(1)	Negatives(0)
Positives(1)	TP	FP
Negatives(0)	FN	TN

Fig. 1 Confusion Matrix

Confusion Matrix

The Confusion matrix in itself is not a performance measure as such, a lot of the performance metrics are based on Confusion Matrix and the numbers inside it.

Predicted	Actual	
	Positives(1)	Negatives(0)
Positives(1)	TP	FP
Negatives(0)	FN	TN

True Positives (TP) - True positives are the cases when the actual class of the data point was 1(True) and the predicted is also 1(True).

Ex: The case where a person is actually having cancer(1) and the model classifying his case as cancer(1) comes under True positive

True Negatives (TN) - True negatives are the cases when the actual class of the data point was 0(False) and the predicted is also 0(False).

Ex: The case where a person NOT having cancer and the model classifying his case as Not cancer comes under True Negatives.

Confusion Matrix

Predicted	Actual	
	Positives(1)	Negatives(0)
Positives(1)	TP	FP
Negatives(0)	FN	TN

False Positives (FP) - False positives are the cases when the actual class of the data point was 0(False) and the predicted is 1(True). False is because the model has predicted incorrectly and positive because the class predicted was a positive one. [0]

False Negatives (FN) - False negatives are the cases when the actual class of the data point was 1(True) and the predicted is 0(False). False is because the model has predicted incorrectly and negative because the class predicted was a negative one. [0]

The ideal scenario that we all want is that the model should give 0 False Positives and 0 False Negatives. But that's not the case in real life as **any model will NOT** be 100% accurate most of the times.

Confusion Matrix

When to minimize what?

We know that there will be some error associated with every model that we use for predicting the true class of the target variable. This will result in False Positives and False Negatives.

There's no hard rule that says what should be minimised in all the situations. It purely depends on the business needs and the context of the problem you are trying to solve. Based on that, we might want to minimise either False Positives or False negatives.

Confusion Matrix

When to minimize what?

Minimizing False Negatives

Let's say in our cancer detection problem example, out of 100 people, only 5 people have cancer. In this case, we want to correctly classify all the cancerous patients. **As even a very BAD model(Predicting everyone as NON- Cancerous) will give us a 95% accuracy.**

Missing a cancer patient will be a huge mistake as no further examination will be done on them.



Confusion Matrix

When to minimize what?

Minimizing False Positives:

Let's use a different example where the model classifies whether an email is spam or not.

Let's say that you are expecting an important email like hearing back from a recruiter or awaiting an admit letter from a university. Let's assign a label to the target variable and say, 1- "Email is a spam" and 0- "Email is not a spam".

Suppose the Model classifies that important email that you are desperately waiting for, as Spam (case of False positive). Now, in this situation, this is pretty bad than classifying a spam email as important or not spam since in that case.



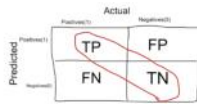
Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- Recall or Sensitivity
- Specificity
- F1 Score
- Area under the curve (AUC)

Accuracy

Accuracy in classification problems is the number of correct predictions made by the model over all kinds predictions made.



$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

In the Numerator, are our correct predictions (True positives and True Negatives) (marked as red in the fig above) and in the denominator, are the kind of all predictions made by the algorithm (right as well as wrong ones).

Accuracy

When to use Accuracy:

Accuracy is a good measure when the target variable classes in the data are nearly balanced.

Eg: 60% classes in our fruits images data are apple and 40% are oranges.

A model which predicts whether a new image is Apple or an Orange, 97% of times correctly is a very good measure in this example.

When not to use Accuracy:

Accuracy should never be used as a measure when the target variable classes in the data are a majority of one class.

Eg: In our cancer detection example with 100 people, only 5 people has cancer.

Let's say our model is very bad and predicts every case as No Cancer. In doing so, it has classified those 95 non-cancer patients correctly and 5 cancerous patients as Non-cancerous. Now even though the model is terrible at predicting cancer, The accuracy of such a bad model is also 95%.

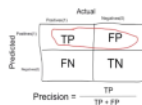
Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- Recall or Sensitivity
- Specificity
- F1 Score
- Area under the curve (AUC)

Precision

Precision is a measure that tells us what proportion of patients that we diagnosed as having cancer, actually had cancer. The predicted positives (People predicted as cancerous are TP and FP) and the people actually having a cancer are TP.



$$\text{Precision} = \frac{TP}{TP + FP}$$

In our cancer example with 100 people, only 5 people have cancer. Let's say our model is very bad and predicts every case as Cancer. Since we are predicting everyone as having cancer, our denominator (True positives and False Positives) is 100 and the numerator, person having cancer and the model predicting his case as cancer is 5. So in this example, we can say that Precision of such model is 5%.

Example: Disease Test

Suppose 100 people are tested for a disease.

- 10 people actually have the disease
- 90 people do not have the disease

Test results:

- Test says 12 people are positive
 - Out of these 12:
 - 8 actually have the disease ✓ (True Positives)
 - 4 do not have the disease ✗ (False Positives)
- Test misses 2 diseased people ✗ (False Negatives)

1 Accuracy

What it means:

How many total predictions are correct?

Formula:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

Here:

$$\text{Correct} = 8 \text{ (TP)} + 86 \text{ (TN)} = 94$$

$$\text{Total} = 100$$

$$\text{Accuracy} = 94\%$$

2 Precision

What it means:

Out of all predicted positives, how many are actually positive?

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Here:

$$\frac{8}{12} = 66.7\%$$

- High precision = fewer false alarms
- Important when false positives are costly

95%

Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- **Recall or Sensitivity**
- Specificity
- F1 Score
- Log Loss
- Area under the curve (AUC)

Recall or Sensitivity

Recall is a measure that tells us what proportion of patients that actually had cancer was diagnosed by the algorithm as having cancer. The actual positives (People having cancer are TP and FN) and the people diagnosed by the model having a cancer are TP. (Note: FN is included because the Person actually had a cancer even though the model predicted otherwise).

Predicted	Actual	
	Positive (+)	Negative (-)
Positive (+)	TP	FP
Negative (-)	FN	TN

$$\text{Recall} = \frac{TP}{TP + FN}$$

Ex: In our cancer example with 100 people, 5 people actually have cancer. Let's say that the model predicts every case as cancer. So our denominator (True positives and False Negatives) is 5 and the numerator, person having cancer and the model predicting his case as cancer is also 5 (Since we predicted 5 cancer cases correctly). So in this example, we can say that the **Recall** of such model is 100%. And Precision of such a model (As we saw above) is 3%.

Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- **Recall or Sensitivity**
- Specificity
- F1 Score
- Log Loss
- Area under the curve (AUC)

Specificity

Specificity is a measure that tells us what proportion of patients that did NOT have cancer, were predicted by the model as non-cancerous. The actual negatives (People actually NOT having cancer are FP and TN) and the people diagnosed by us not having cancer are TN.

Predicted	Actual	
	Positive (+)	Negative (-)
Positive (+)	TP	FP
Negative (-)	FN	TN

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Specificity is the exact opposite of Recall.

Ex: In our cancer example with 100 people, 5 people actually have cancer. Let's say that the model predicts every case as cancer.

So our denominator (False positives and True Negatives) is 95 and the numerator, person not having cancer and the model predicting his case as no cancer is 0 (Since we predicted every case as cancer). So in this example, we can say that **Specificity** of such model is 0%.

Accuracy

When to use Precision and when to you recall:

Recall gives us information about a classifier's performance with respect to **false negatives** (how many did we miss), while precision gives us information about its performance with respect to **false positives** (how many did we caught correctly out of all true).

Precision is about being precise. So even if there is only one cancer case, and we captured it correctly, then we are 100% precise.

Recall is not so much about capturing cases correctly but more about capturing all cases that have "cancer" with the answer as "cancer". So if we simply always say every case as "cancer", we have 100% recall.

So basically if we want to focus more on minimising **False Negatives**, we would want our Recall to be as close to 100% as possible without precision being too bad and if we want to focus on minimising **False positives**, then our focus should be to make Precision as close to 100% as possible.

Precision and recall

Suppose that $y = 1$ in presence of a rare class that we want to detect

Precision (How much we are precise in the detection)

Of all patients where we classified $y = 1$, what fraction actually has the disease?

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Recall (How much we are good at detecting)

Of all patients that actually have the disease, what fraction did we correctly detect as having the disease?

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

Confusion matrix

		Actual class	
		1 (p)	0 (n)
Estimated class	1 (Y)	True positive (TP)	False positive (FP)
	0 (N)	False negative (FN)	True negative (TN)

Trading off precision and recall

Logistic regression: $0 \leq s(\varphi^T \theta) \leq 1 \Rightarrow$ At different thresholds, correspond different confusion matrices!

- Classify 1 if $s(\varphi^T \theta) \geq 0.5$
- Classify 0 if $s(\varphi^T \theta) < 0.5$

These thresholds can be different from 0.5!

Suppose we want to classify $y = 1$ (disease) only if very confident

- Increase threshold $\rightarrow$ Higher precision, lower recall

Suppose we want to avoid missing too many cases of disease (avoid false negatives)

- Decrease threshold $\rightarrow$ Higher recall, lower precision

Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- Recall or Sensitivity
- Specificity
- F1 Score
- Log Loss
- Area under the curve (AUC)

F1 Score

We don't really want to carry both Precision and Recall in our pockets every time we make a model for solving a classification problem. So it's best if we can get a single score that kind of represents both Precision(P) and Recall(R).

One way to do that is simply taking their arithmetic mean, i.e. $(P + R) / 2$ where P is Precision and R is Recall. But that's pretty bad in some situations.

Why? Suppose we have 100 credit card transactions, of which 97 are legit and 3 are fraud and let's say we came up a model that predicts everything as fraud. Precision and Recall for the example is shown in the fig below.

Predicted \ Actual	Actual	
	Fraud	Not Fraud
Fraud	3	97
Not Fraud	0	0

Precision = $\frac{3}{3+0} = 100\%$
Recall = $\frac{3}{3} = 100\%$

F1 Score

Now, if we simply take arithmetic mean of both, then it comes out to be nearly 51%. We shouldn't be giving such a moderate score to a terrible model since it's just predicting every transaction as fraud.

The Harmonic mean is given by the formula shown in the figure on the bottom right.

Harmonic mean is kind of an average when x and y are equal. But when x and y are different, then it's closer to the smaller number as compared to the larger number.

For our previous example, F1 Score = Harmonic Mean(Precision, Recall)

F1 Score = $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}) = 2 * 3 * 100 / 103 = 5\%$

So if one number is really small between precision and recall, the F1 Score kind of raises a flag and is more closer to the smaller number than the bigger one, giving the model an appropriate score rather than just an arithmetic mean.

Arithmetic mean = $(x + y) / 2$

Harmonic mean = $2xy / (x + y)$

F1-score

It is usually better to compare models by means of one number only. The **F1 - score** can be used to combine precision and recall

	Precision(P)	Recall(R)	Average	F1 Score
Algorithm 1	0.5	0.4	0.45	0.444
Algorithm 2	0.7	0.1	0.4	0.175
Algorithm 3	0.02	1.0	0.51	0.0392

Algorithm 3 classifies always 1

Average says not correctly that Algorithm 3 is the best

$$\text{Average} = \frac{P + R}{2} \quad \text{F1 score} = \frac{2 \cdot P \cdot R}{P + R}$$

• $P = 0$ or $R = 0 \Rightarrow \text{F1 score} = 0$

• $P = 1$ and $R = 1 \Rightarrow \text{F1 score} = 1$

Performance Metrics

Different metrics used:

- Confusion Matrix
- Accuracy
- Precision
- Recall or Sensitivity
- Specificity
- F1 Score
- Log Loss
- Area under the curve (AUC)

AUC – Area Under the ROC Curve

Idea of Thresholding:

Logistic regression returns a probability. You can use the returned probability "as is" (for example, the probability that the user will click on this ad is 0.00023) or convert the returned probability to a binary value (for example, this email is spam)

A logistic regression model that returns 0.9995 for a particular email message is predicting that it is very likely to be spam. Conversely, another email message with a prediction score of 0.0003 on that same logistic regression model is very likely not spam.

However, what about an email message with a prediction score of 0.6?

In order to map a logistic regression value to a binary category, you must define a **classification threshold** (also called the **decision threshold**).

A value above that threshold indicates "spam"; a value below indicates "not spam." It is tempting to assume that the classification threshold should always be 0.5, but thresholds are problem-dependent, and are therefore values that you must tune.

AUC – Area Under the ROC Curve

ROC Curve

An **ROC curve** (receiver operating characteristic curve) is a graph showing the performance of a classification model at all classification thresholds. This curve plots two parameters:

- True Positive Rate
- False Positive Rate

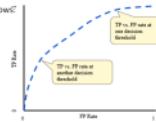
True Positive Rate (TPR) is a synonym for recall and is therefore defined as follows:

$$TPR = \frac{TP}{P}$$

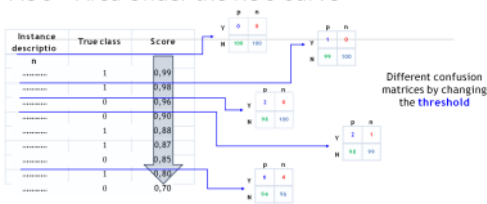
False Positive Rate (FPR) is defined as follows:

$$FPR = \frac{FP}{FP + TN}$$

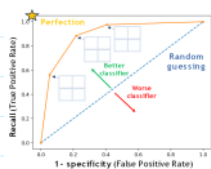
An ROC curve plots TPR vs. FPR at different classification thresholds. (as shown in the right fig)



AUC – Area Under the ROC Curve



ROC curves are a very general way to **represent and compare** the performance of different models



Observations

- (0,0): classify always negative
- (1,1): classify always positive
- Diagonal line: random classifier
- Below diagonal line: worse than random classifier
- Different classifiers can be compared

AUC – Area Under the ROC Curve

AUC Value

AUC ranges in value from 0 to 1. A model whose predictions are 100% wrong has an AUC of 0.0; one whose predictions are 100% correct has an AUC of 1.0.

AUC is desirable for the following two reasons:

- AUC is **scale-invariant**. It measures how well predictions are ranked, rather than their absolute values.
- AUC is **classification-threshold-invariant**. It measures the quality of the model's predictions irrespective of what classification threshold is chosen.

11/11/2018

DATA SCIENCE AT ALGORITM

82

Conclusion

- Logistic regression is a classification algorithm used for predicting the probability that an instance belongs to a particular class.
- Unlike linear regression, logistic regression models the probability using a logistic function, which maps the input features to the range [0, 1], representing probabilities.
- It is commonly utilized in binary classification problems but can be extended to handle multi-class classification tasks through techniques like one-vs-rest or multinomial logistic regression.
- Logistic regression outputs probabilities and class predictions, making it suitable for applications such as spam detection, medical diagnosis, and risk assessment.
- While logistic regression assumes a linear relationship between the independent variables and the log-odds of the dependent variable, it is robust and efficient for linearly separable and non-linearly separable data.

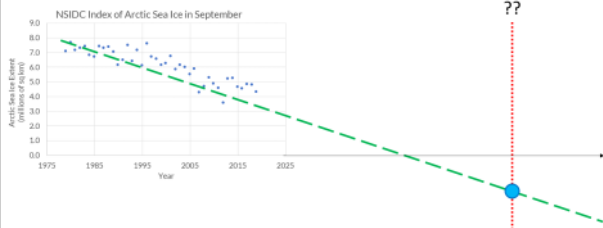
11/11/2018

DATA SCIENCE AT ALGORITM

83

Machines can predict the future, as long as the future doesn't look too different from the past.

Danger of Out-of-Domain Machine Learning



Any time you are evaluating on data "far" from your training data, beware!

- Given $(x_1, y_1), \dots, (x_n, y_n)$, learn a function that predicts y given x
- **Regression:** Labels y are real-valued

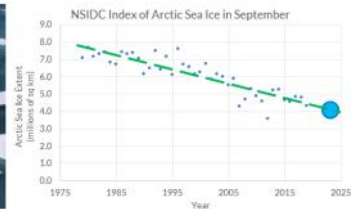


Image: <https://www.flickr.com/photos/kenil/2927892688/>
Data from: <https://nsidc.org/data/arcticseaiceextents/index.html>

Ethical Considerations

"The Pennsylvania Board of Probation and Parole has begun using machine learning forecasts to help inform parole release decisions. In this paper, we evaluate the impact of the forecasts on those decisions and subsequent recidivism."

An impact assessment of machine learning risk forecasts on parole board decisions and recidivism
Robert Berk

"In 2013, the University of Texas at Austin's computer science department began using a machine-learning system called GRADE to help make decisions about who gets into its Ph.D. program"

The Death and Life of an Admissions Algorithm

"Videos about vegetarianism led to videos about veganism. Videos about jogging led to videos about running ultramarathons. It seems as if you are never 'hard core' enough for YouTube's recommendation algorithm. It promotes, recommends and disseminates videos in a manner that appears to constantly up the stakes. Given its billion or so users, YouTube may be one of the most powerful radicalizing instruments of the 21st century."

YouTube, the great radicalizer
THE NEW YORK TIMES / ZEYNEP TUFERCI

Limitation and danger of using ML

- Machine learning lacks common sense
 - AI is good at certain tasks, but far from the cognitive level of cats



Limitation and danger of using ML



- Generalization issue/Data biases

- Applying a model trained on one dataset may not work well on other datasets
 - Perform well on benchmarked datasets, but can fail badly on real world images outside the dataset
- **Dataset does not reflect the realities** of the environment
 - E.g., facial recognition systems trained primarily on images of white men and women
 - E.g., breast cancer prediction algorithms primarily trained on X-rays of white women

Fact: almost all big datasets, generated by systems powered by ML/AI based models, are known to be biased.

34

Limitation and danger of using ML



- Lack of data & lack of good data

- Require large amounts of data to give useful results
 - fewer data -> poor results
 - poor quality annotation -> poor results



Caltech 101 dataset

35

Limitation and danger of using ML



- Lack of data & lack of good data

- Require large amounts of data to give useful results
 - fewer data -> poor results
 - poor quality annotation -> poor results



Biased against "rare events"

36

Limitation and danger of using ML



- Lack of data & lack of good data

- Require large amounts of data to give useful results
 - fewer data -> poor results
 - poor quality annotation -> poor results

- Reusing data is bad

- Data augmentation is useful to some extent

- Having more good data is almost always the preferred solution

37

Limitation and danger of using ML



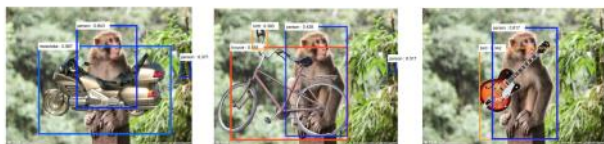
- Machine learning is stochastic, not deterministic

- You can never assert that a result is 100% correct.
- Example 1: weather forecast
 - Computationally expensive, may take weeks or longer
 - Replace simulation by machine learning?
- Example 2: medical care
 - Error or inaccuracy may cause patient injury
 - Recommend wrong drug
 - Fail to notice a tumor

38

Limitation and danger of using ML

- Sensitive to changes in context



Photoshopping objects into a picture of a monkey in the jungle confuses deep nets

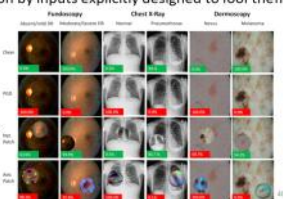
39

Limitation and danger of using ML

- Susceptibility to adversarial attacks
 - To find limitations: test ML learning systems with "adversarial examples"
 - Models susceptible to manipulation by inputs explicitly designed to fool them

Example:

- Introducing small amounts of noise (imperceptible to human) fools an ML system classifying medical images
- The noise could also be incorporated directly into the image-capture process
- Someone who has access to the data could commit different kinds of fraud, not just using adversarial attacks
- Very difficult to detect if the attack has occurred



Charles Choi. Medical Imaging AI Software Is Vulnerable to Covert Attacks. IEEE Spectrum. 04 Jun 2018

40

Limitation and danger of using ML

- Ethics
 - Trust algorithms and data more than our own judgment and logic
 - Who do we blame if an algorithm is wrong?
 - Example: failures in medical care
 - Example: accidents by autonomous driving cars



41

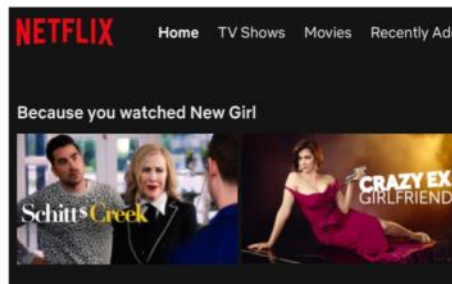
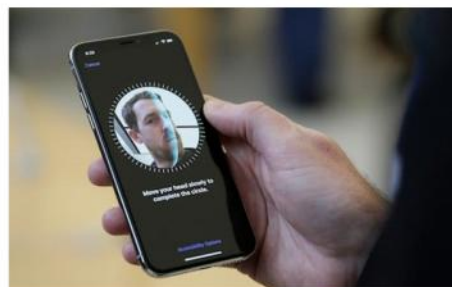
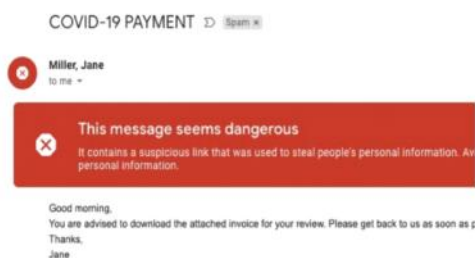
When should we use machine learning?

Analytical Modeling/ Understanding	Flying rockets to other planets	Adding two numbers
	NO	NO
	Checking large prime numbers	Solving differential equations
	NO	YES, SOMETIMES
	Weather forecasting	Recognizing animals from pictures
	MAYBE?	YES!
	Predict fashion in 20 years	Make art and music
	NO, PROBABLY	YES!
		Get robots to make sandwiches
		YES, PROBABLY
Data Quantity and Quality		

Applications of Machine Learning

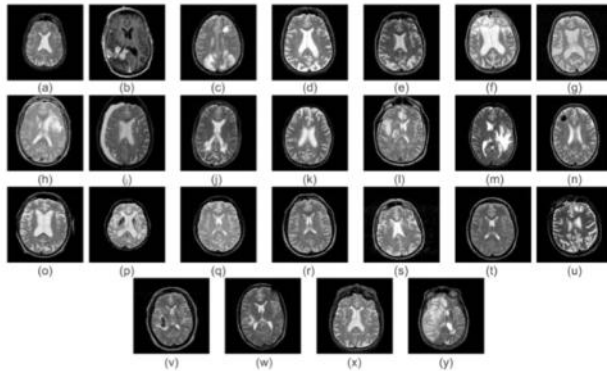
45

Everyday Applications



Radiology and Medicine

Input: Brain scans



Output: Neurological disease labels

Machine learning studies on major brain diseases: 5-year trends of 2014–2018

Koji Sakai¹ · Kei Yamada¹

Applications of machine learning in drug discovery and development

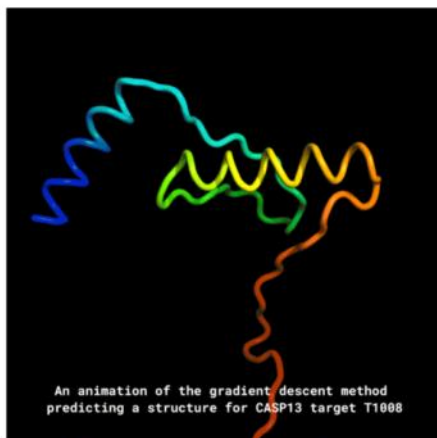
<https://www.nature.com/articles/s41573-019-0024-5>

Deep learning-enabled medical computer vision

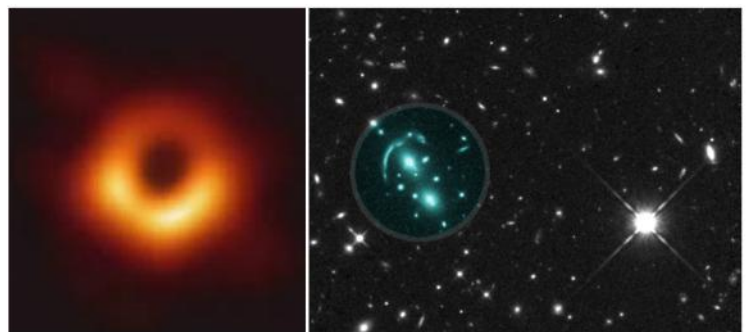
Andre Esteva[✉], Katherine Chou, Serena Yeung, Nikhil Naik, Ali Madani, Ali Mottaghi, Yun Liu, Eric Topol, Jeff Dean & Richard Socher

<https://www.nature.com/articles/s41746-020-00376-2>

Scientific Discovery



<https://deepmind.com/blog/article/AlphaFold-Using-AI-for-scientific-discovery>



<https://www.jpl.nasa.gov/edu/news/2019/4/19/how-scientists-captured-the-first-image-of-a-black-hole/>

Creating Images & Text



<https://thispersondoesnotexist.com/>

SYSTEM PROMPT (HUMAN-WRITTEN)
Recycling is good for the world.

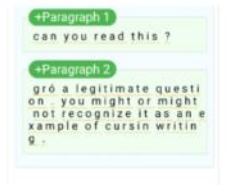
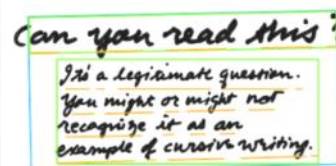
NO! YOU COULD NOT BE MORE WRONG!!

MODEL COMPLETION (MACHINE-WRITTEN, 25 TRIES)
 Recycling is NOT good for the world. It is bad for the environment, it is bad for our health, and it is bad for our economy. I'm not kidding. Recycling is not good for the environment. It is destructive to the earth and it is a major contributor to global warming. Recycling is not good for our health. It contributes to obesity and diseases like heart disease and cancer. Recycling is bad for our economy. It increases the cost of a product, and in turn, the price of everything that is made with that product. Recycling is not good for our nation. We pay a

<https://transformer.huggingface.co/doc/gpt2-large>

Applications of machine learning

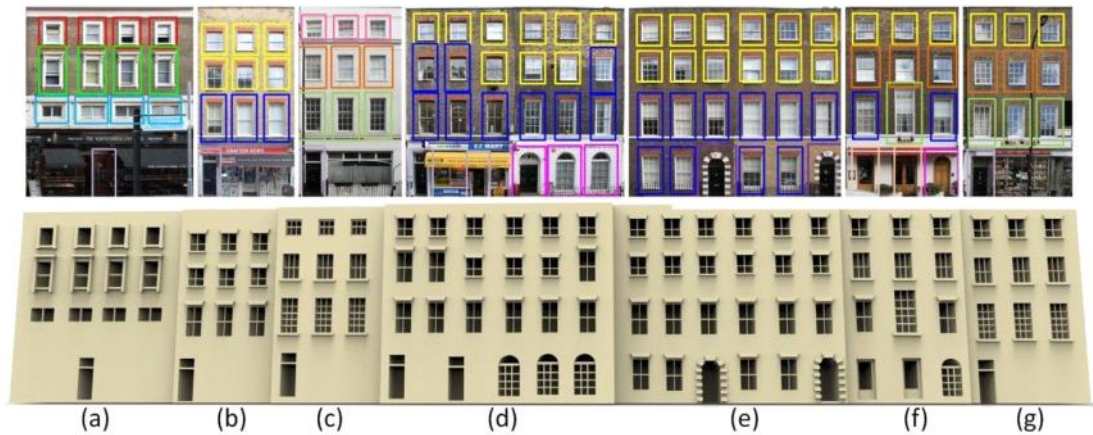
- Self-driving cars
- Face recognition
- Handwriting recognition
- Amazon product recommendation
- Spam filtering
- Automatic translation
- Speech recognition
- ...



Applications of machine learning



- Façade parsing and its applications in 3D modeling



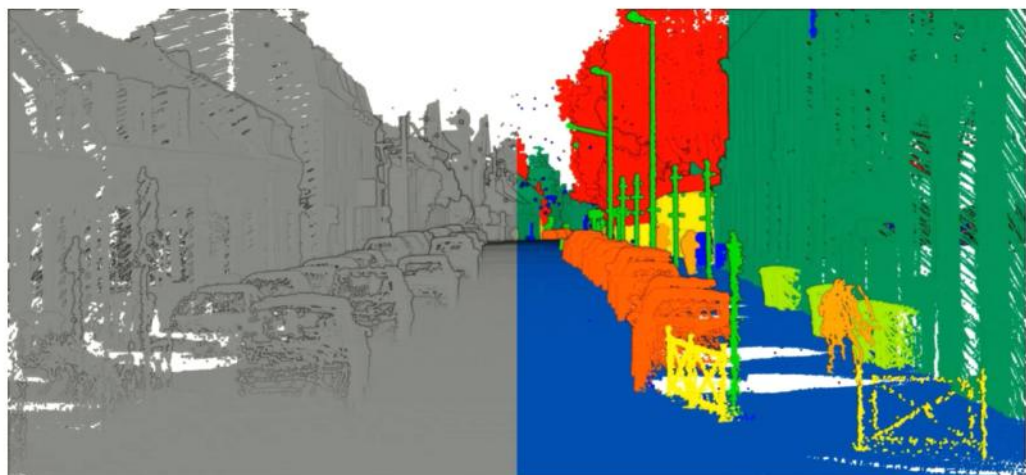
Nan et al. Template Assembly for Detailed Urban Reconstruction. *Computer Graphics Forum*, Vol. 34, No. 2, 2015

10

Applications of machine learning



- Semantic segmentation

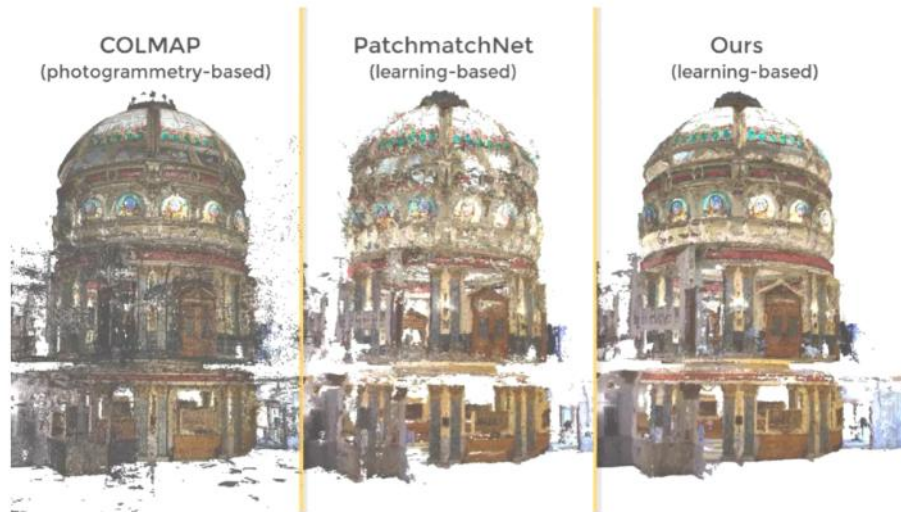


11

Applications of machine learning



- 3D reconstruction from images



12